

Interview Questions & Answers

Full-Stack Development & QA Engineering

18 Topics • 200+ Questions with Expert Answers

HTML, CSS, JavaScript, Java, MySQL, Angular, Spring Framework, Spring Boot, Maven, Git, GitHub, Selenium, TestNG, RestAssured, SoapUI, Cucumber, Jenkins, Docker

Topics Covered

01. HTML
02. CSS
03. JavaScript
04. Java
05. MySQL
06. Angular
07. Spring Framework
08. Spring Boot
09. Maven
10. Git
11. GitHub
12. Selenium
13. TestNG
14. RestAssured
15. SoapUI
16. Cucumber
17. Jenkins
18. Docker

1. HTML

◆ Basics & Fundamentals

Q1. What is HTML and what does it stand for?

■ HTML stands for HyperText Markup Language. It is the standard language used to create and structure content on the web. HTML uses elements represented by tags to define the structure and meaning of web content such as headings, paragraphs, links, and images.

Q2. What is the difference between HTML and HTML5?

■ HTML5 is the latest major version of HTML, introducing semantic elements (article, section, header, footer, nav), native audio/video without plugins, the canvas element, localStorage/sessionStorage APIs, new form input types (date, email, range, color), geolocation, and Web Workers. Older HTML lacked these and relied heavily on Flash plugins.

Q3. What are block-level vs inline elements? Give examples.

■ Block-level elements start on a new line and take full available width: div, p, h1-h6, ul, ol, table, form. Inline elements flow within surrounding text, only as wide as needed: span, a, strong, em, img, input, label. Inline elements cannot contain block-level elements in standard HTML.

Q4. Explain the structure of an HTML document.

■ An HTML document begins with `<!DOCTYPE html>`, then the `<html>` root element containing `<head>` (metadata: title, meta tags, CSS links, scripts — not displayed) and `<body>` (all visible page content). Every valid HTML page must have this basic skeleton to render correctly across browsers.

Q5. What is the purpose of the DOCTYPE declaration?

■ `<!DOCTYPE html>` tells the browser which version of HTML is used so it renders the page correctly in standards mode. Without it, browsers may enter 'quirks mode' and render pages inconsistently. In HTML5 the declaration is simply `<!DOCTYPE html>`, replacing the lengthy HTML4 declarations.

Q6. What are void/self-closing elements? Give examples.

■ Void elements cannot have child content and need no closing tag: `<br>`, `<hr>`, `<img>`, `<input>`, `<link>`, `<meta>`, `<area>`, `<base>`, `<col>`. In XHTML they are written as `<br />`, but in HTML5 the trailing slash is optional. Attempting to nest content inside them is invalid.

Q7. What are semantic HTML elements and why are they important?

■ Semantic elements clearly describe their meaning to browsers and developers: `<header>`, `<nav>`, `<main>`, `<article>`, `<section>`, `<aside>`, `<footer>`, `<figure>`, `<time>`. They improve SEO by giving search engines clear content signals, enhance accessibility for screen readers, and make code more readable and maintainable.

Q8. What is the difference between section, article, aside, and main?

■ `<article>` is self-contained, redistributable content (blog post, news story). `<section>` groups related content with a heading, part of a larger context. `<aside>` holds tangentially related content (sidebars, pull quotes). `<main>` identifies the primary content of the page — only one per page. These distinctions matter for accessibility tools and SEO crawlers.

Q9. What are HTML entities? Give examples.

■ HTML entities are special codes to display reserved or special characters: `&` for `&`, `<` for `<`, `>` for `>`, ` ` for non-breaking space, `©` for copyright symbol, `"` for quotation mark. They prevent reserved characters from being interpreted as HTML markup by the browser.

Q10. What is the difference between strong and b, em and i?

■ `<strong>` indicates strong importance (semantic) and renders bold; `<b>` is purely presentational bold with no semantic weight. `<em>` indicates stress emphasis (semantic) and renders italic; `<i>` is purely presentational italic. Screen readers may alter tone for `<strong>/<em>` but treat `<b>/<i>` as plain text — use semantic versions for accessibility.

◆ Forms & Input

Q11. What are HTML forms and how do you create one?

■ Forms collect user input and submit it to a server. Created with `<form action='/submit' method='POST'>`. Inside: input elements (`<input>`, `<textarea>`, `<select>`), labels, and a submit button. The action attribute is the server URL; method is GET or POST. The browser serializes form data and sends it on submit.

Q12. What is the difference between GET and POST methods?

■ GET appends form data as URL query parameters (`?name=value`) — visible, bookmarkable, cached, limited in size. POST sends data in the request body — not visible in URL, not cached, supports large payloads and binary data. Use POST for sensitive data (passwords), file uploads, or data that modifies server state.

Q13. What are the different input types introduced in HTML5?

■ HTML5 added: email (validates email format), number, range (slider), date, time, datetime-local, month, week, color (color picker), search, tel, url. These provide built-in validation and appropriate mobile keyboards (e.g., numeric pad for number), reducing the need for custom JavaScript validation.

Q14. What is the purpose of the label element?

■ `<label>` provides a text description for a form control. When associated via the for attribute (matching the input's id) or by wrapping the input, clicking the label focuses the control. Screen readers announce the label when the control is focused — essential for accessibility compliance (WCAG).

Q15. How do you make a form field required in HTML5?

■ Add the required attribute to the input: `<input type='text' name='username' required>`. On form submission, the browser validates the field is not empty and displays a native error message. Combine with the pattern attribute for regex validation. Server-side validation is still essential as client-side can be bypassed.

Q16. What is the enctype attribute and when do you use multipart/form-data?

■ enctype specifies how form data is encoded on submission. Default is `application/x-www-form-urlencoded` (key=value URL-encoded pairs). When uploading files with `<input type='file'>`, you must use `enctype='multipart/form-data'`, which splits data into parts allowing binary files to be transmitted. `text/plain` is a third rarely-used option.

Q17. What is the datalist element?

■ `<datalist>` provides an autocomplete suggestion list for an `<input>` linked via the list attribute matching the datalist's id. Users can still type free-form text not in the list, unlike `<select>`. Example: `<input list='colors'><datalist id='colors'><option value='Red'><option value='Blue'></datalist>`. Combines free input with convenience suggestions.

◆ Media, Tables & Advanced

Q18. How do you embed audio and video in HTML5?

■ Use `<audio>` and `<video>` tags with `src` or nested `<source>` elements for multiple formats: `<video controls><source src='v.mp4' type='video/mp4'><source src='v.webm' type='video/webm'>Fallback text</video>`. Key attributes: `controls`, `autoplay`, `loop`, `muted`, `poster` (thumbnail). Multiple sources let the browser choose a supported format.

Q19. What is the canvas element used for?

■ `<canvas>` provides a bitmap drawing surface manipulated via JavaScript using the 2D Canvas API or WebGL for 3D. Used for games, charts, image processing, and animations. Unlike SVG, canvas is raster-based (pixel operations) with no DOM for drawn shapes — you must re-draw on change. Example: `ctx.fillRect(10,10,100,100)`.

Q20. What are `thead`, `tbody`, `tfoot` in a table?

■ `<thead>` groups header rows (column titles), `<tbody>` contains data rows, `<tfoot>` contains footer rows (totals). These improve accessibility (screen readers identify header vs data), allow independent `tbody` scrolling, and enable browsers to repeat `<thead>/<tfoot>` when printing multi-page tables. Optional but best practice.

Q21. What is the difference between `colspan` and `rowspan`?

■ `colspan` merges a cell across multiple columns horizontally: `<td colspan='3'>` spans 3 columns. `rowspan` merges across multiple rows vertically: `<td rowspan='2'>` spans 2 rows. When using these, you must omit the cells that the spanning cell occupies from subsequent rows/columns to avoid layout breakage.

Q22. What is an `iframe` and what are its security concerns?

■ `<iframe>` embeds another HTML page inside the current page. Security concerns include clickjacking (malicious overlay to capture clicks), cross-frame scripting, and data theft from untrusted content. Mitigations: use the `sandbox` attribute to restrict permissions, set `X-Frame-Options` and `Content-Security-Policy` headers, and only embed trusted sources.

Q23. What do the `async` and `defer` attributes do on a script tag?

■ Both allow HTML parsing to continue while the script downloads. `async` executes immediately when downloaded — order not guaranteed, good for independent scripts like analytics. `defer` executes after HTML parsing is complete, in document order — ideal for scripts needing the DOM. Without either, parsing halts during download and execution.

Q24. What is the difference between `localStorage`, `sessionStorage`, and cookies?

■ `localStorage` persists data indefinitely across tabs, up to ~5MB, client-side only. `sessionStorage` stores data only for the current tab session (cleared on tab close), ~5MB, client-side only. Cookies are sent with every HTTP request to the server, 4KB limit, support expiry dates, `HttpOnly` and `Secure` flags. Use `localStorage/sessionStorage` for client state; cookies for server-side sessions and authentication tokens.

Q25. What are `data-*` attributes?

■ Custom data attributes (`data-*`) store extra information on HTML elements accessible via JavaScript. Example: `<button data-user-id='42' data-role='admin'>`. Access: `element.dataset.userId` returns `'42'`. Useful for passing config from HTML to JS without hidden inputs or globals. Accessible in CSS via `[data-role='admin']` attribute selectors.

Q26. What is ARIA and how do aria-label, aria-hidden, and role help accessibility?

■ ARIA (Accessible Rich Internet Applications) provides attributes to improve accessibility for screen reader users. `aria-label` gives a text label when visible label text is insufficient (icon buttons). `aria-hidden='true'` hides decorative elements from screen readers. `role` defines semantic purpose (`role='dialog'`, `role='alert'`, `role='button'`). Together they bridge gaps when native HTML semantics are insufficient for complex UI.

2. CSS

◆ Selectors, Specificity & Cascade

Q1. What is CSS specificity and how is it calculated?

■ Specificity determines which CSS rule applies when multiple target the same element. Calculated as four values: inline styles (1,0,0,0), IDs (0,1,0,0), classes/attributes/pseudo-classes (0,0,1,0), elements/pseudo-elements (0,0,0,1). The highest total wins. Equal specificity uses source order (last rule wins). `!important` overrides all specificity.

Q2. What are pseudo-classes and pseudo-elements?

■ Pseudo-classes select elements in a specific state with a single colon: `:hover`, `:focus`, `:active`, `:nth-child()`, `:first-child`, `:checked`, `:not()`. Pseudo-elements select and style part of an element with double colons: `::before`, `::after`, `::first-line`, `::first-letter`, `::placeholder`, `::selection`. Pseudo-elements create virtual elements in the DOM without changing HTML.

Q3. What is the difference between `:nth-child()` and `:nth-of-type()`?

■ `:nth-child(n)` counts all sibling elements regardless of type — `p:nth-child(2)` selects a `<p>` only if it is the 2nd child overall. `:nth-of-type(n)` counts only siblings of the same tag — `p:nth-of-type(2)` selects the 2nd `<p>` among siblings, skipping other element types. The distinction matters in mixed-element containers.

Q4. How does the CSS cascade work?

■ The cascade determines which declaration wins when multiple apply. Priority order (highest to lowest): 1) `!important` declarations, 2) inline styles, 3) specificity of selector, 4) source order (later wins if specificity is equal). Origin also matters: browser defaults < user styles < author styles, with `!important` reversing the origin order.

Q5. What does `!important` do and when should you avoid it?

■ `!important` forces a declaration to apply regardless of specificity or order: `color: red !important`. Avoid it because it breaks the natural cascade, makes debugging very difficult, and leads to specificity wars. Acceptable uses: overriding third-party styles you cannot modify, and utility classes that must always apply. It should be a last resort.

Q6. What are CSS combinators?

■ Combinators define relationships between selectors. Descendant (space): `'div p'` — all `p` inside any `div`. Child (`>`): `'div > p'` — only direct `p` children of `div`. Adjacent sibling (`+`): `'h1 + p'` — first `p` immediately after `h1`. General sibling (`~`): `'h1 ~ p'` — all `p` siblings after `h1`. Understanding combinators allows writing precise, efficient selectors.

Q7. How does inheritance work in CSS?

■ Some CSS properties inherit from parent to child by default: `color`, `font-family`, `font-size`, `line-height`, `text-align`, `visibility`. Layout properties (`margin`, `padding`, `border`, `background`, `width`) do not inherit. Control inheritance with: `inherit` (force inheritance), `initial` (browser default), `unset` (`inherit` if inheritable, else `initial`), `revert` (cascade to user-agent style).

◆ Box Model & Layout

Q8. Explain the CSS box model.

- Every HTML element is a rectangular box with four layers from inside out: Content (text/image area), Padding (transparent space between content and border), Border (surrounds the padding), Margin (transparent space outside the border separating elements from each other). Width/height set by default apply only to the content area (content-box model).
-

Q9. What is the difference between box-sizing content-box and border-box?

- content-box (default): width/height apply to content area only — padding and border are added on top, making the total element larger than specified. border-box: width/height include content, padding, and border — total box stays at the specified size. Most developers use `* { box-sizing: border-box }` globally for more intuitive sizing.
-

Q10. What is margin collapse?

- Margin collapse occurs when top and bottom margins of adjacent block elements merge into a single margin equal to the largest value (not the sum). Happens between: adjacent siblings, parent and first/last child (if no border/padding separates them), and empty blocks. Horizontal margins never collapse. Prevented by overflow: hidden, display: flex/grid, or adding padding/border to the parent.
-

Q11. How does CSS Flexbox work?

- Flexbox is a one-dimensional layout system. Set display: flex on the container; direct children become flex items. Container properties: flex-direction (row/column), justify-content (main axis), align-items (cross axis), flex-wrap, gap. Item properties: flex-grow, flex-shrink, flex-basis, align-self, order. Excellent for distributing space and aligning items along one axis.
-

Q12. What is the difference between justify-content and align-items?

- justify-content aligns items along the main axis (horizontal in flex-direction: row): flex-start, flex-end, center, space-between, space-around, space-evenly. align-items aligns along the cross axis (vertical in row direction): flex-start, flex-end, center, stretch (default), baseline. When flex-direction is column, the axes swap.
-

Q13. How does CSS Grid work?

- CSS Grid is a two-dimensional layout system. Set display: grid on the container. Define columns with grid-template-columns (e.g., '1fr 2fr') and rows with grid-template-rows. Items are placed in cells; use grid-column/grid-row to span multiple cells. The fr unit distributes available space fractionally. gap controls spacing between cells. Ideal for complex page layouts.
-

Q14. What is the difference between Flexbox and Grid?

- Flexbox is one-dimensional — lays out items in a row OR column, working from content outward. Grid is two-dimensional — manages rows AND columns simultaneously, working from layout inward. Use Flexbox for navigation bars, toolbars, and single-axis components. Use Grid for full-page layouts and complex multi-column/row designs needing precise control in both dimensions.
-

Q15. Explain the position property.

- static (default): normal document flow. relative: offset from its normal position; still occupies original space. absolute: removed from flow, positioned relative to nearest non-static ancestor. fixed: removed from flow, positioned relative to viewport — stays during scroll. sticky: relative until a scroll threshold, then fixed — used for sticky headers/sidebars.
-

Q16. What is z-index and how does stacking context work?

■ z-index controls the stack order of overlapping elements (higher = on top). Only works on positioned elements (relative/absolute/fixed/sticky) or flex/grid items. A stacking context is created by position+z-index, opacity<1, transform, filter, isolation: isolate, etc. Child z-index values are relative to their context — a child cannot appear above a sibling's stacking context regardless of its z-index value.

◆ Responsive Design & Advanced

Q17. What are CSS media queries?

■ Media queries conditionally apply CSS based on device/viewport characteristics. Syntax: @media (max-width: 768px) { body { font-size: 14px; } }. Conditions include width, height, orientation (portrait/landscape), resolution, and media type (screen, print). They are the foundation of responsive web design for adapting layouts to different screen sizes.

Q18. What is mobile-first design?

■ Mobile-first means writing base CSS for mobile screens first, then using min-width media queries to add styles for larger screens. This ensures mobile performance (less CSS to parse), forces prioritization of core content, and results in cleaner code. It contrasts with desktop-first (starting with desktop styles and using max-width queries to scale down).

Q19. What are CSS variables?

■ CSS custom properties (variables) store reusable values: :root { --primary: #1a237e; }. Used with var(): color: var(--primary). Scoped (can be overridden in child selectors), support inheritance, and can be updated via JavaScript at runtime. Unlike preprocessor variables, they are live — changes propagate immediately without recompilation.

Q20. What is the difference between em, rem, px, vh, and vw?

■ px is an absolute pixel unit. em is relative to the current element's font-size — compounds in nested elements. rem (root em) is relative to the root html font-size — no compounding, preferred for font sizing. vh is 1% of viewport height; vw is 1% of viewport width — great for full-screen layouts. Use rem for typography, px for borders/shadows, vh/vw for viewport-relative sizing.

Q21. How do CSS transitions and animations work?

■ Transitions smoothly interpolate a property between two states triggered by a state change (hover): transition: background-color 0.3s ease. Animations use @keyframes for multi-step sequences: @keyframes slide { from { left: 0 } to { left: 100px } } applied with animation: slide 1s ease infinite. Transitions need a trigger; animations are self-triggering and can loop.

Q22. What is the difference between transition and animation?

■ Transitions require a triggering state change and interpolate between two states — simpler. Animations with @keyframes are self-triggering, can have multiple intermediate steps (0%, 50%, 100%), can loop, alternate direction, and control fill-mode (what happens before/after). Animations are more powerful for complex, multi-step motion sequences.

Q23. How do you center an element horizontally and vertically?

■ Flexbox (modern preferred): parent { display: flex; justify-content: center; align-items: center; }. Grid: parent { display: grid; place-items: center; }. Absolute + transform: child { position: absolute; top: 50%; left: 50%; transform: translate(-50%, -50%); }. For horizontal only: margin: 0 auto (block with set width). text-align: center for inline content.

Q24. What is the difference between visibility hidden and display none?

■ display: none removes the element from document flow entirely — no space taken, not rendered, not accessible. visibility: hidden hides the element visually but it still occupies its layout space (invisible box). With visibility, child elements can override with visibility: visible. Use display: none to completely remove from layout; visibility: hidden to hide while preserving space.

Q25. What are CSS preprocessors like SASS/SCSS?

■ CSS preprocessors extend CSS with programming features. SASS/SCSS adds: variables (\$primary: #333), nesting (selectors inside selectors), mixins (@mixin/@include for reusable style blocks), functions, partials (@use), inheritance (@extend), and operators. SCSS is a superset of CSS (valid CSS is valid SCSS). They compile to plain CSS. LESS is another popular preprocessor.

Q26. What is BEM methodology?

■ BEM (Block Element Modifier) is a CSS naming convention for scalable class names. Block: standalone component (card, menu). Element: part of a block using double underscore (card__title, menu__item). Modifier: variant using double hyphen (card--featured, button--large). Prevents naming conflicts, makes relationships clear, and improves CSS maintainability in large projects.

3. JavaScript

◆ Core Concepts

Q1. What is the difference between var, let, and const?

■ var is function-scoped, hoisted and initialized as undefined, can be re-declared. let is block-scoped, hoisted but not initialized (temporal dead zone causes ReferenceError if accessed early), cannot be re-declared in same scope. const is block-scoped like let, cannot be re-assigned (though object properties can mutate). Use const by default, let when reassignment is needed, avoid var.

Q2. What is hoisting in JavaScript?

■ Hoisting moves variable and function declarations to the top of their scope during compilation. Function declarations are fully hoisted (callable before declaration). var declarations are hoisted and initialized as undefined. let and const are hoisted but not initialized — accessing them before declaration throws a ReferenceError (temporal dead zone). Class declarations are also in the temporal dead zone.

Q3. What is the difference between == and ===?

■ === (strict equality) compares value AND type without coercion: 5 === '5' is false. == (loose equality) coerces types before comparison: 5 == '5' is true, null == undefined is true, 0 == false is true. Always prefer === to avoid counterintuitive coercion bugs. The only acceptable use of == is null == null to check for both null and undefined at once.

Q4. What is the difference between null and undefined?

■ undefined means a variable was declared but not assigned, a function returns nothing, or a property doesn't exist. null is an intentional absence of value — explicitly set by a developer to indicate 'no value here'. typeof undefined is 'undefined'; typeof null is 'object' (a historical JavaScript bug). null == undefined is true but null === undefined is false.

Q5. What is type coercion?

■ Type coercion is JavaScript's automatic conversion between types. Implicit: '5' + 3 = '53' (string concatenation wins), '5' - 3 = 2 (numeric subtraction). Boolean('') is false; Boolean('hello') is true. Explicit: Number(), String(), Boolean(), parseInt(), parseFloat(). Understanding coercion is critical for avoiding subtle bugs, especially in conditional checks and comparisons with ==.

Q6. What is the difference between function declaration and expression?

■ Function declaration: function greet() {} — fully hoisted, callable before its position in code. Function expression: const greet = function() {} — only the variable is hoisted (as undefined), calling before definition throws TypeError. Named function expressions help with debugging (name appears in stack traces). Arrow functions are always expressions. Use declarations for top-level utilities, expressions for callbacks.

Q7. What are arrow functions and how do they differ?

■ Arrow functions (=>) provide shorter syntax and crucially do NOT have their own this — they inherit this from the enclosing lexical context. They also have no arguments object, cannot be constructors (no new), and have no prototype. Ideal for callbacks and array methods. Avoid as object methods when this should refer to the object, or when you need the arguments object.

Q8. What is a closure?

■ A closure is a function that retains access to its outer scope variables even after the outer function has returned. Example: `function counter() { let n=0; return () => ++n; }` — the inner function closes over `n`. Closures enable data encapsulation, factory functions, and memoization. Every function in JavaScript is technically a closure.

Q9. What is the scope chain?

■ The scope chain is the hierarchy JavaScript searches when resolving a variable: current scope first, then outer scopes up to the global scope. If not found anywhere, a `ReferenceError` is thrown. The chain is determined at function definition time (lexical scoping), not call time — this is what makes closures possible. Each function creates a new scope level.

Q10. What are truthy and falsy values?

■ Exactly 8 falsy values: `false`, `0`, `-0`, `0n`, `''` (empty string), `null`, `undefined`, and `NaN`. Everything else is truthy — including `'0'` (non-empty string), `[]` (empty array), `{}` (empty object), and `-1`. This matters in `if (value)` checks. Common bug: `if (arr.length)` works, but `if (arr)` is always true even for empty arrays.

◆ Objects, Arrays & Prototypes

Q11. What is the prototype chain?

■ Every JavaScript object has an internal `[[Prototype]]` link to another object (or `null`). When accessing a property, JS first checks the object itself, then its prototype, continuing up the chain until found or `null` is reached. `Object.prototype` is at the top of most chains. `Object.getPrototypeOf(obj)` inspects the chain. This mechanism implements inheritance in JavaScript.

Q12. What is destructuring in JavaScript?

■ Destructuring unpacks values from arrays or properties from objects: Array: `const [a, b] = [1, 2]`. Object: `const { name, age } = person`. Supports defaults (`const { x = 0 } = obj`), renaming (`const { name: fullName } = obj`), nested destructuring, and rest patterns (`const [first, ...rest] = arr`). Widely used in function parameters, React props, and API response handling.

Q13. What is the spread operator and rest parameter?

■ Spread `(...)` expands an iterable into elements: `Math.max(...arr)`, `[...arr1, ...arr2]`, `{...obj1, ...obj2}`. Rest `(...)` collects remaining elements/arguments into an array: `function fn(first, ...others)`. Rest must be the last parameter. Same syntax, opposite directions: spread expands at call sites; rest collects in definitions.

Q14. What are common array methods?

■ `map()` transforms each element into a new array. `filter()` creates a new array with elements passing a test. `reduce()` accumulates to a single value. `find()` returns the first matching element (or `undefined`). `findIndex()` returns the index. `some()` returns true if any element passes. `every()` returns true if all pass. All are non-mutating. These functional methods replace most traditional for-loop patterns.

Q15. What is the difference between `map()` and `forEach()`?

■ `map()` returns a new array of transformed values — use it when you need the result. `forEach()` executes a callback for side effects and returns `undefined` — use for logging or DOM updates. You cannot break out of `forEach` (use `for...of` instead). `map()` chains with other array methods; `forEach()` does not. Rule: if you need the result, use `map()`; if you only have side effects, use `forEach()`.

Q16. What is the difference between `slice()` and `splice()`?

■ `slice(start, end)` returns a shallow copy of a portion without mutating the original — like Python slicing. `splice(start, deleteCount, ...items)` modifies the array in place, removing and optionally inserting elements, and returns removed elements. Memory: `sLice = cLone` (non-mutating), `sPLice = sPLice` into place (mutating). Confusing them is a common bug.

Q17. What is deep copy vs shallow copy?

■ A shallow copy duplicates only top-level properties — nested objects still share references with the original. Methods: `Object.assign()`, spread operator, `slice()`. A deep copy duplicates everything recursively with no shared references. Methods: `structuredClone()` (modern built-in), `JSON.parse(JSON.stringify(obj))` (loses functions/Date/undefined), or `Lodash _.cloneDeep()`. Use deep copy when modifying nested objects must not affect the original.

Q18. What are getters and setters?

■ Getters and setters define computed properties and intercept property access/assignment. `get property() { return this._val * 2 }` — called on access without parentheses. `set property(val) { this._val = val }` — called on assignment. Defined with `get/set` in object literals or classes. Enable data encapsulation, validation on set, lazy computation on get, and virtual/computed properties.

◆ Asynchronous JavaScript

Q19. What is the event loop?

■ The event loop makes JavaScript's single-threaded model handle async operations. The call stack runs synchronous code. When async operations complete, callbacks go to the microtask queue (Promises) or macro-task queue (`setTimeout`, `setInterval`). The loop: when the call stack is empty, drain ALL microtasks, then process ONE macro task, then repeat. This is why `Promise.then()` always runs before `setTimeout` even with 0ms delay.

Q20. What are callbacks and callback hell?

■ A callback is a function passed as an argument, called after an async operation completes. Callback hell (pyramid of doom) occurs when callbacks nest deeply: `readFile(function(data) { parse(data, function(result) { save(result, function() { ... }) }) })`. Creates poor readability, tangled error handling, and maintenance nightmares. Solved by Promises and `async/await`.

Q21. What are Promises?

■ A Promise represents a value that may be available now or in the future. Three states: pending (initial), fulfilled (resolved with value — `.then()` called), rejected (failed — `.catch()` called). Created with `new Promise((resolve, reject) => {})`. States are immutable once settled. Promises enable chaining (`.then().then()`) and cleaner error handling than callbacks. `.finally()` runs regardless of outcome.

Q22. What is the difference between `Promise.all`, `race`, `allSettled`, and `any`?

■ `Promise.all([p1,p2])`: resolves when ALL fulfill (array of results); rejects immediately if any reject. `Promise.race([p1,p2])`: resolves/rejects with the FIRST to settle. `Promise.allSettled([p1,p2])`: resolves when all settle (fulfilled or rejected), never rejects — returns array with status for each. `Promise.any([p1,p2])`: resolves with the first fulfilled; rejects only if ALL reject. `allSettled` is safest when you need all results.

Q23. What is `async/await`?

■ `async/await` is syntactic sugar over Promises. An `async` function always returns a Promise. `await` pauses execution inside the `async` function until the Promise resolves — without blocking the main thread. Errors caught with `try/catch` instead of `.catch()`. Makes `async` code read like synchronous code. Under the hood, it is still Promise-based — you can mix `.then()` and `await`.

Q24. What is the difference between microtask and macro-task queues?

■ Microtasks (higher priority): Promise callbacks (.then/.catch/.finally), queueMicrotask(), MutationObserver. After each task, ALL pending microtasks run before the next macro task. Macro tasks: setTimeout, setInterval, setImmediate, I/O events, UI rendering. Result: Promise.resolve().then(fn) runs before setTimeout(fn, 0) even with 0ms delay, because microtasks drain completely before the next macro task.

Q25. How do you make HTTP requests with fetch?

■ fetch(url) returns a Promise resolving to a Response. Critically, fetch only rejects on network errors — a 404 still resolves! Always check res.ok or res.status. Example: fetch('/api/data').then(res => { if(!res.ok) throw new Error(res.status); return res.json(); }).then(data => console.log(data)). For POST: fetch(url, { method:'POST', headers:{'Content-Type':'application/json'}, body: JSON.stringify(data) }).

◆ DOM, Events & ES6+

Q26. What is event bubbling and event capturing?

■ Events propagate in three phases: Capturing (window down to target), Target (event at the target element), Bubbling (target up to window). Listeners use bubbling by default. Pass true as third argument to addEventListener for capturing. stopPropagation() stops further propagation. stopImmediatePropagation() also prevents other listeners on the same element. Event delegation relies on bubbling.

Q27. What is event delegation?

■ Event delegation attaches ONE listener to a parent instead of many listeners on individual children. Since events bubble, the parent catches all child events. Check event.target to identify which child triggered it. Benefits: fewer listeners (better performance), automatically handles dynamically added children. Example: ul.addEventListener('click', e => { if(e.target.tagName==='LI') handleClick(e.target) }).

Q28. What is the difference between innerHTML, innerText, and textContent?

■ innerHTML gets/sets HTML including tags — XSS risk if setting untrusted content. innerText returns only visible text (respects CSS, hidden elements excluded), triggers layout reflow when read. textContent returns all text including hidden elements, faster (no layout reflow), does not parse HTML. For reading: prefer textContent. For setting safe text: prefer textContent to avoid XSS. Only use innerHTML for intentional HTML from trusted sources.

Q29. What are template literals?

■ Template literals use backticks (`) and support: embedded expressions \${expression}, multi-line strings without \n, and tagged templates (calling a function with the literal parts and values). They eliminate string concatenation with +, support readable multi-line strings, and enable advanced patterns like styled-components CSS-in-JS and SQL query builders via tagged templates.

Q30. What are JavaScript modules?

■ ES Modules (ESM) split code into files with explicit dependencies. Named exports: export const PI = 3.14; export function fn() {} — imported as import { PI, fn } from './module.js'. Default export: export default class App {} — imported as import App from './module.js'. Modules are always strict mode, execute once (cached), and are lazily evaluated. Use type='module' on script tags in HTML. CommonJS (require/module.exports) is used in Node.js.

Q31. What is the difference between call, apply, and bind?

■ All three control the this value inside a function. call(thisArg, arg1, arg2) invokes immediately with args listed individually. apply(thisArg, [arg1, arg2]) invokes immediately with args as an array — useful when args are already in array form. bind(thisArg, arg1) returns a NEW function with this permanently bound, not invoked — useful for event handlers. Memory: call = comma-separated, apply = array.

Q32. What is debouncing and throttling?

■ Debouncing ensures a function fires only AFTER a specified delay following the LAST call — ideal for search-as-you-type (wait until user stops typing). Throttling ensures a function fires at most ONCE per specified interval regardless of call frequency — ideal for scroll/resize handlers. Both optimize performance by limiting function call rates. Example: `debounce(fn, 300)` fires 300ms after the last keystroke.

Q33. What is CORS?

■ CORS (Cross-Origin Resource Sharing) is a browser security mechanism restricting web pages from requesting a different origin (domain, protocol, or port). Servers must include `Access-Control-Allow-Origin` headers to permit cross-origin requests. Preflight `OPTIONS` requests are sent for non-simple requests. CORS is browser-enforced — server-to-server requests are unaffected. Solutions: configure server CORS headers, use a proxy server, or JSONP (legacy).

Q34. What is an IIFE?

■ An IIFE (Immediately Invoked Function Expression) is defined and executed immediately: `(function() { })()` or `(() => {} )()`. Creates a local scope, preventing variable pollution of the global scope. Widely used before ES6 modules for module patterns and jQuery plugins. Today less necessary due to block scoping with `let/const` and native ES modules, but still useful for isolated initialization code.

Q35. What are Symbols in JavaScript?

■ Symbol is a primitive type that creates unique, immutable identifiers. `Symbol('desc') !== Symbol('desc')` — every Symbol is unique even with the same description. Not enumerable in `for...in` or `Object.keys()`, invisible to `JSON.stringify()`. Used as unique object property keys to avoid naming collisions in libraries, and as well-known symbols (`Symbol.iterator`, `Symbol.hasInstance`) to customize built-in language behaviors.

4. Java

◆ OOP Concepts

Q1. What are the four pillars of OOP?

- Encapsulation: bundling data and methods, restricting direct access via private fields and getters/setters.
- Abstraction: hiding implementation complexity, exposing only necessary interface (abstract classes, interfaces).
- Inheritance: acquiring properties/methods from a parent class using extends, promoting code reuse.
- Polymorphism: same interface works on different types — overloading (compile-time) and overriding (runtime).

Q2. What is the difference between abstraction and encapsulation?

- Abstraction hides complexity and shows only relevant features: a Car exposes drive() without revealing engine internals. Encapsulation hides internal state via private fields with controlled access through getters/setters. Abstraction is a design concept (WHAT); encapsulation is an implementation mechanism (HOW). Both achieve information hiding but at different levels.

Q3. Why does Java not support multiple inheritance through classes?

- Java avoids multiple class inheritance to prevent the Diamond Problem — ambiguity when two parents define the same method and a child inherits from both (unclear which to use). Java solves this by allowing only single class inheritance but supporting multiple interface implementation. Java 8+ default methods can still cause diamond issues, resolved by overriding in the implementing class.

Q4. What is method overloading vs method overriding?

- Overloading (compile-time polymorphism): same method name with different parameter lists in the SAME class — resolved at compile time based on argument types. Overriding (runtime polymorphism): subclass provides its own implementation of a parent's method with the same signature — resolved at runtime via dynamic dispatch. Overriding requires @Override annotation (recommended), cannot reduce visibility, and cannot override final/static/private methods.

Q5. What is the difference between abstract class and interface?

- Abstract class can have instance variables, constructors, concrete and abstract methods, any access modifiers — supports single inheritance only. Interface can have constants, abstract methods, default/static/private methods (Java 8+/9+) — supports multiple implementation. Use abstract class when sharing common state/behavior. Use interface to define contracts for multiple unrelated classes. A class can extend one class but implement many interfaces.

Q6. What is final in Java?

- final class cannot be subclassed (e.g., String, Integer — for immutability/security). final method cannot be overridden in subclasses, ensuring consistent behavior. final variable is a constant — must be initialized once, cannot be reassigned (for primitives: value constant; for objects: reference constant but state can change). static final creates a class constant: static final int MAX = 100.

Q7. What is the difference between static and instance methods?

- Static methods belong to the class, called on the class (Math.sqrt()), cannot access instance variables or use this, loaded with class loading. Instance methods belong to objects, can access both instance and static members, require object creation before calling. Static methods are used for utility/helper functions, factory methods, and operations not requiring object state.

Q8. What is constructor chaining?

■ Constructor chaining calls one constructor from another. Within the same class: use `this(...)` as the first statement. To call parent class: use `super(...)` as the first statement. Reduces code duplication when multiple constructors share initialization logic. If neither `this()` nor `super()` is explicitly called, the compiler inserts `super()` to call the no-arg parent constructor. All chains end at `Object()`.

◆ Core Java

Q9. What is the difference between JVM, JRE, and JDK?

■ JVM (Java Virtual Machine) is the runtime engine executing Java bytecode — platform-specific, provides memory management and garbage collection. JRE (Java Runtime Environment) = JVM + standard class libraries — needed to RUN Java programs. JDK (Java Development Kit) = JRE + compiler (javac) + debugger + dev tools — needed to DEVELOP Java programs. To run apps: install JRE. To develop: install JDK.

Q10. What is the difference between `==` and `.equals()` in Java?

■ `==` compares references (memory addresses) for objects — checks if two variables point to the same object. For primitives, `==` compares values. `.equals()` compares object content/state — overridden in `String`, `Integer`, `List` to provide meaningful equality. `new String('Hi') == new String('Hi')` is false (different objects), but `.equals()` returns true (same content). Always use `.equals()` for objects; `==` only for primitives or null checks.

Q11. What is the difference between `String`, `StringBuilder`, and `StringBuffer`?

■ `String` is immutable — every operation creates a new object, causing overhead in loops. `StringBuilder` is mutable and NOT thread-safe — use in single-threaded contexts for building strings. `StringBuffer` is mutable and thread-safe (synchronized) — use in multi-threaded scenarios. `StringBuilder` is preferred over `StringBuffer` when thread safety is not needed (much faster). Example: `StringBuilder sb = new StringBuilder(); sb.append('Hello'); sb.toString();`

Q12. What is String interning?

■ Java maintains a `String` pool where literals are cached. `String s = 'Hello'` checks the pool first and reuses existing. `String.intern()` manually adds to the pool and returns the pooled reference. `String` literals with `==` may return true if they reference the same pooled object. `new String('Hello')` bypasses the pool and creates a heap object. This explains surprising `==` results with strings in Java.

Q13. What is autoboxing and unboxing?

■ Autoboxing is automatic conversion of a primitive to its wrapper class: `int` to `Integer`, `double` to `Double`. Unboxing is the reverse. Happens implicitly: `List<Integer> list = new ArrayList<>(); list.add(5)` — `int 5` is autoboxed to `Integer`. Performance caution: autoboxing in tight loops causes many object allocations. `NullPointerException` risk: unboxing a null `Integer` throws `NPE`. Use primitives in performance-critical code.

Q14. What is the difference between checked and unchecked exceptions?

■ Checked exceptions (compile-time) extend `Exception` (not `RuntimeException`) — compiler forces handling via `try-catch` or `throws` declaration. Examples: `IOException`, `SQLException`. Unchecked exceptions extend `RuntimeException` — no compiler enforcement. Examples: `NullPointerException`, `ArrayIndexOutOfBoundsException`. Use checked for recoverable conditions (file not found), unchecked for programming errors (null pointer). Errors (`OutOfMemoryError`) should not be caught.

Q15. What is try-with-resources?

■ Try-with-resources (Java 7+) automatically closes `AutoCloseable` resources at the end of the `try` block, even if an exception occurs: `try (Connection conn = getConn()) { ... }`. Resources closed in reverse declaration order. Eliminates finally blocks for cleanup. If both the `try` block and `close()` throw, the `close` exception is suppressed and accessible via `getSuppressed()`. Essential for JDBC connections and I/O streams.

Q16. What is garbage collection in Java?

■ GC automatically reclaims memory occupied by unreachable objects. The JVM heap has: Young Generation (Eden + Survivor spaces for new objects), Old Generation (long-lived objects), and Metaspace (class metadata). GC algorithms: Serial, Parallel, G1 (default), ZGC. Minor GC collects Young Gen; Major/Full GC collects all generations. `System.gc()` is only a hint — you cannot force GC. Avoid memory leaks by nulling references when done.

Q17. What are Java generics?

■ Generics enable type-safe code: `List<String>` ensures only Strings are stored, eliminating `ClassCastException` at runtime. Generics are erased at runtime (type erasure) — bytecode uses `Object`. Generic methods: `<T> T method(T arg)`. Bounded parameters: `<T extends Comparable<T>>`. Wildcards: `?` (unknown), `? extends T` (upper bound, read-safe), `? super T` (lower bound, write-safe). PECS principle: Producer Extends, Consumer Super.

Q18. What is the difference between throw and throws?

■ `throw` is a statement that explicitly raises an exception: `throw new IllegalArgumentException('Invalid')`. Used inside a method body. `throws` is a declaration in the method signature indicating the method may throw checked exceptions: `public void readFile() throws IOException`. `throws` is a warning to callers; `throw` is the actual action. Multiple exceptions: `throws IOException, SQLException`. For unchecked exceptions, `throws` is optional but documenting them is good practice.

◆ Collections Framework

Q19. What is the difference between List, Set, and Map?

■ List is an ordered collection allowing duplicates, elements accessed by index: `ArrayList`, `LinkedList`. Set is a collection disallowing duplicates, generally unordered (`LinkedHashSet` preserves insertion order, `TreeSet` sorts): `HashSet`, `TreeSet`. Map stores unique key-value pairs, keys unique: `HashMap`, `LinkedHashMap`, `TreeMap`. Map is not a true Collection. Choose List for ordered data with duplicates, Set for uniqueness, Map for key-based lookup.

Q20. What is the difference between ArrayList and LinkedList?

■ `ArrayList` uses a dynamic array. $O(1)$ random access by index, $O(n)$ insert/delete in the middle (shifting). Best for frequent reads and appends. `LinkedList` uses a doubly-linked list. $O(n)$ access by index, $O(1)$ insert/delete at head/tail when you have the node. Best for frequent insertions at ends. `LinkedList` also implements `Queue` and `Deque`. In practice, `ArrayList` is almost always faster due to better CPU cache locality.

Q21. What is the difference between HashMap, LinkedHashMap, and TreeMap?

■ `HashMap`: unordered, allows one null key, $O(1)$ average get/put — best general-purpose map. `LinkedHashMap`: maintains insertion order (or access order for LRU caches), otherwise like `HashMap`. `TreeMap`: sorted by natural key order or `Comparator`, $O(\log n)$ operations — implements `NavigableMap` with `firstKey()`, `floorKey()`. Use `HashMap` for speed, `LinkedHashMap` for ordered iteration, `TreeMap` for sorted keys.

Q22. How does HashMap work internally?

■ `HashMap` uses an array of `Node<K,V>` (buckets). On `put(key,value)`: compute `key.hashCode()`, hash to a bucket index. If bucket is empty, add node. Collision: nodes form a linked list, or red-black tree if size > 8 (Java 8+). On `get(key)`: same hash finds bucket, then `equals()` finds exact key. Default capacity 16, load factor 0.75 — at 75% full, capacity doubles (rehashing). Keys must consistently implement `hashCode()` and `equals()`.

Q23. What is the difference between HashMap and Hashtable?

■ Hashtable is synchronized (thread-safe) but slower. HashMap is not synchronized but faster for single-threaded use. Hashtable forbids null keys and values; HashMap allows one null key and multiple null values. Hashtable is legacy (Java 1.0); HashMap is part of the Collections Framework (Java 1.2). For thread safety prefer ConcurrentHashMap over Hashtable. For single-threaded use, always prefer HashMap. Hashtable is essentially obsolete.

Q24. What is ConcurrentHashMap?

■ ConcurrentHashMap provides thread-safe operations without locking the entire map. In Java 8+, it uses node-level locking (CAS + synchronized on individual bins) allowing concurrent reads (non-blocking) and concurrent writes to different segments. Does not allow null keys or values. Atomic methods: compute(), merge(), forEach(), putIfAbsent(). Much faster than Hashtable/Collections.synchronizedMap() for concurrent access. The go-to thread-safe map.

Q25. What is fail-fast vs fail-safe iterator?

■ Fail-fast iterators throw ConcurrentModificationException if the collection is structurally modified during iteration. They operate directly on the collection — ArrayList, HashMap iterators. Fail-safe iterators work on a snapshot copy, so no exception on modification during iteration, but may not reflect changes made after iterator creation — CopyOnWriteArrayList, ConcurrentHashMap iterators. Fail-safe has higher memory overhead but is safer for concurrent use.

Q26. How do Comparable and Comparator work?

■ Comparable defines natural ordering within the class by implementing compareTo(T o) — the class itself must implement Comparable<T>. String, Integer implement Comparable. Collections.sort(list) uses this. Comparator is an external strategy implementing compare(T o1, T o2) — useful when you can't modify the class or need multiple sort orders. Java 8: Comparator.comparing(Person::getAge).thenComparing(Person::getName).reversed() for fluent chaining.

◆ Java 8+ & Multithreading

Q27. What are lambda expressions?

■ Lambdas provide concise syntax for implementing functional interfaces (single abstract method). Syntax: (params) -> expression or (params) -> { statements; }. Example: list.sort((s1, s2) -> s1.compareTo(s2)). They enable functional programming, eliminate anonymous class boilerplate, and integrate seamlessly with the Stream API. Lambdas can capture effectively-final local variables from the enclosing scope.

Q28. What is the Stream API?

■ The Stream API (Java 8+) enables functional-style processing of element sequences. Sources: Collections, arrays, generators. Intermediate operations (lazy, return Stream): filter, map, sorted, distinct, limit. Terminal operations (eager, trigger processing): collect, count, reduce, forEach, findFirst. Streams are consumed once, do not modify the source, and can be parallel (parallelStream()). Example: list.stream().filter(x -> x > 0).map(String::valueOf).collect(Collectors.toList()).

Q29. What is the difference between map() and flatMap() in streams?

■ map() transforms each element to one result — Stream<List<String>> stays Stream<List<String>>. flatMap() maps each element to a stream then flattens all into one — Stream<List<String>> becomes Stream<String>. Use flatMap when your mapping function returns a collection you want to flatten. Example: orders.stream().flatMap(o -> o.getItems().stream()) gives a flat stream of all items across all orders.

Q30. What is Optional in Java?

■ `Optional<T>` (Java 8+) is a container that may or may not contain a non-null value, designed to reduce `NullPointerException`s. Created with `Optional.of(value)` (NPE if null), `Optional.ofNullable(value)` (safe), `Optional.empty()`. Key methods: `isPresent()`, `isEmpty()`, `get()` (unsafe), `orElse(default)`, `orElseGet(supplier)`, `orElseThrow()`, `map()`, `flatMap()`, `filter()`. Best used as return type for methods that may return nothing. Avoid as field type or method parameter.

Q31. What are method references?

■ Method references are shorthand for lambdas that delegate to an existing method. Four types: Static: `Math::sqrt` (equivalent to `x -> Math.sqrt(x)`). Instance of specific object: `System.out::println`. Instance of arbitrary object of a type: `String::toUpperCase` (equivalent to `s -> s.toUpperCase()`). Constructor: `ArrayList::new`. More readable than lambdas when no transformation of arguments is needed.

Q32. What is the difference between extending Thread and implementing Runnable?

■ Extending Thread: `class MyThread extends Thread { public void run() {} }` — since Java supports single inheritance only, this prevents extending other classes. Implementing Runnable: `class MyTask implements Runnable { public void run() {} }` — preferred: separates task from execution mechanism, allows extending other classes, and the same Runnable can be submitted to thread pools. With Runnable: `new Thread(new MyTask()).start()` or `executor.submit(task)`.

Q33. What is the synchronized keyword?

■ `synchronized` ensures only one thread at a time executes a block or method, preventing race conditions. Synchronized method: `public synchronized void increment()` — locks on the instance. Synchronized block: `synchronized(lockObject) { ... }` — more granular, better performance. The lock releases when the block exits (normally or via exception). Overuse causes thread contention and performance bottlenecks. Prefer `java.util.concurrent` classes for more sophisticated concurrency.

Q34. What is a deadlock and how do you prevent it?

■ Deadlock: two or more threads permanently blocked, each waiting for a resource held by another. Example: Thread A holds Lock 1, waits for Lock 2; Thread B holds Lock 2, waits for Lock 1. Prevention: lock ordering (always acquire locks in the same global order), lock timeout (`ReentrantLock.tryLock` with timeout), avoid nested locks, use higher-level concurrent utilities (`java.util.concurrent`), and prefer lock-free structures like `ConcurrentHashMap`.

Q35. What is the volatile keyword?

■ `volatile` ensures a variable is always read from and written to main memory, not from a thread's CPU cache — guaranteeing visibility across threads. It does NOT guarantee atomicity for compound operations (`i++` is read-modify-write, not atomic). Use `volatile` for simple boolean flags where one thread writes and others read. For compound atomic operations, use `AtomicInteger/AtomicLong`. `volatile` also prevents instruction reordering by the JVM/CPU.

Q36. What is the Executor framework?

■ The Executor framework (`java.util.concurrent`) provides a higher-level replacement for directly creating threads. `ExecutorService` manages a thread pool. Implementations: `Executors.newFixedThreadPool(n)`, `newCachedThreadPool()`, `newSingleThreadExecutor()`. Submit tasks with `execute(Runnable)` or `submit(Callable)` (returns `Future<T>`). Always call `shutdown()` when done. `ScheduledExecutorService` handles delayed/periodic tasks. Prevents thread-creation overhead and caps resource usage.

5. MySQL

◆ SQL Fundamentals

Q1. What is the difference between DDL, DML, DCL, and TCL?

■ DDL (Data Definition Language) defines schema: CREATE, ALTER, DROP, TRUNCATE — auto-committed. DML (Data Manipulation Language) manipulates data: SELECT, INSERT, UPDATE, DELETE — can be rolled back. DCL (Data Control Language) controls access: GRANT, REVOKE. TCL (Transaction Control Language) manages transactions: COMMIT, ROLLBACK, SAVEPOINT. DDL affects structure; DML affects data; TCL controls transaction boundaries.

Q2. What is the difference between WHERE and HAVING?

■ WHERE filters individual rows BEFORE grouping/aggregation — cannot use aggregate functions. HAVING filters groups AFTER GROUP BY — can use aggregate functions. Example: SELECT dept, COUNT(*) FROM emp WHERE salary > 50000 GROUP BY dept HAVING COUNT(*) > 5 — WHERE filters individual employees by salary; HAVING filters departments by headcount. Without GROUP BY, HAVING technically works like WHERE but is bad practice.

Q3. What is the difference between DELETE, TRUNCATE, and DROP?

■ DELETE removes specific rows based on WHERE (DML, row-by-row logging, can be rolled back). Without WHERE, removes all rows but keeps table structure. TRUNCATE removes ALL rows instantly by deallocating data pages (DDL, auto-committed in most databases, resets AUTO_INCREMENT, cannot use WHERE). DROP removes the entire table structure and data permanently. Summary: DELETE = selective, TRUNCATE = fast total wipe, DROP = destroy.

Q4. What are SQL constraints?

■ PRIMARY KEY: uniquely identifies each row, no nulls, one per table (can be composite). FOREIGN KEY: enforces referential integrity by referencing another table's primary/unique key. UNIQUE: all values in column(s) must be distinct (allows one NULL). NOT NULL: prevents null values. CHECK: ensures values satisfy a condition (age > 0). DEFAULT: assigns default value when none provided. Constraints enforce data integrity at the database level, beyond application code.

Q5. What is the difference between INNER JOIN, LEFT JOIN, RIGHT JOIN, and FULL OUTER JOIN?

■ INNER JOIN: rows matching in BOTH tables. LEFT JOIN: all rows from left table + matching rows from right (NULL for non-matches on right). RIGHT JOIN: all rows from right + matching from left (NULL for non-matches on left). FULL OUTER JOIN: all rows from both tables with NULLs where no match (MySQL simulates with UNION of LEFT and RIGHT JOIN). CROSS JOIN: cartesian product of both tables.

Q6. What is a subquery and a correlated subquery?

■ A subquery is a query nested inside another. Non-correlated subquery executes independently and passes the result to the outer query: SELECT * FROM emp WHERE salary > (SELECT AVG(salary) FROM emp). Correlated subquery references the outer query's columns and re-executes for each outer row: SELECT * FROM emp e WHERE salary > (SELECT AVG(salary) FROM emp WHERE dept_id = e.dept_id). Correlated subqueries can be slow; often replaceable with JOINS.

Q7. What is the difference between UNION and UNION ALL?

■ UNION combines results from two SELECT statements and removes duplicate rows (sorts and deduplicates — adds overhead). UNION ALL combines results including all duplicates — faster, no sorting needed. Both require the same number of columns with compatible types. Use UNION ALL when you know there are no duplicates or you want all rows, as it is significantly more efficient. Use UNION only when removing duplicates is required.

Q8. What are aggregate functions?

■ Aggregate functions compute a single result from multiple rows: COUNT(*) counts all rows; COUNT(col) counts non-NULL values; SUM(col) sums values; AVG(col) averages (ignores NULLs); MIN(col) and MAX(col) find extremes; GROUP_CONCAT(col) concatenates values (MySQL). Used with GROUP BY for per-group aggregation. All except COUNT(*) ignore NULL values.

Q9. How does GROUP BY work?

■ GROUP BY groups rows with the same values in specified columns into summary rows, allowing aggregate functions per group. Example: SELECT department, COUNT(*), AVG(salary) FROM employees GROUP BY department. In MySQL's ONLY_FULL_GROUP_BY mode (default), all non-aggregate SELECT columns must appear in GROUP BY. SQL execution order: FROM → WHERE → GROUP BY → HAVING → SELECT → ORDER BY.

◆ Indexes, Performance & Advanced

Q10. What is an index in MySQL and why use it?

■ An index is a data structure (B-tree by default in InnoDB) storing a sorted copy of column(s) with pointers to rows, enabling fast data retrieval. Without an index, MySQL does a full table scan. Indexes speed up SELECT WHERE, ORDER BY, JOIN, and GROUP BY operations. Trade-off: slow down INSERT/UPDATE/DELETE (index must update) and use disk space. Index columns frequently used in WHERE clauses, JOIN conditions, or ORDER BY.

Q11. What is the difference between clustered and non-clustered indexes?

■ Clustered index determines physical row order on disk — in InnoDB the primary key is always the clustered index. Only ONE per table. Data rows live in the clustered index leaf nodes. Non-clustered (secondary) indexes store indexed column values plus a reference to the clustered key — many per table. Primary key reads are fastest (single B-tree lookup). Secondary index reads require two B-tree lookups: secondary then clustered.

Q12. What is normalization? Explain 1NF, 2NF, 3NF.

■ Normalization reduces data redundancy and dependency. 1NF: atomic column values, no repeating groups, each row unique. 2NF: meets 1NF + no partial dependencies (every non-key attribute depends on the ENTIRE composite primary key). 3NF: meets 2NF + no transitive dependencies (non-key columns depend only on the primary key, not on other non-key columns). Higher normal forms (BCNF, 4NF) address more specific anomalies.

Q13. What is a view?

■ A view is a virtual table defined by a stored SELECT query — stores no data itself. Benefits: simplifies complex queries (define once, query as table), security (expose only specific columns/rows), consistency (logic defined once), and backward compatibility. Updatable views allow INSERT/UPDATE/DELETE under specific conditions. Created with CREATE VIEW name AS SELECT ...; queried like a table.

Q14. What is a stored procedure vs a function?

■ Stored procedure: precompiled code called with CALL; can have IN/OUT/INOUT parameters; can return multiple result sets; cannot be used inside SELECT. Function: always returns exactly one value; used inside SQL (SELECT, WHERE); only IN parameters (MySQL); should be side-effect free. Both offer code reuse, reduced network traffic, and encapsulation. Procedures for complex multi-step operations; functions for computed values.

Q15. What is a trigger in MySQL?

■ A trigger is stored SQL that automatically fires in response to INSERT, UPDATE, or DELETE on a table. BEFORE or AFTER the event. Access old/new values with OLD.col and NEW.col. FOR EACH ROW means one execution per affected row. Uses: audit logging, enforcing business rules, maintaining derived data. Triggers add hidden complexity, make debugging harder, and can impact performance — use judiciously.

Q16. What are ACID properties?

■ Atomicity: transaction is all-or-nothing — any failure rolls back the entire transaction. Consistency: transaction brings database from one valid state to another, enforcing all rules. Isolation: concurrent transactions execute as if serial — intermediate states invisible to others. Durability: committed transactions persist even after system failure (written to disk/WAL). InnoDB is ACID-compliant; MyISAM is not (no transaction support).

Q17. What are isolation levels in MySQL?

■ READ UNCOMMITTED: can read uncommitted changes (dirty reads) — fastest, least safe. READ COMMITTED: reads only committed data — prevents dirty reads, allows non-repeatable reads. REPEATABLE READ (InnoDB default): prevents dirty and non-repeatable reads — same SELECT returns same data in a transaction (InnoDB uses MVCC/gap locks to prevent phantom reads too). SERIALIZABLE: strictest, full locking, prevents all anomalies but slowest. Set with SET TRANSACTION ISOLATION LEVEL.

Q18. What is the difference between CHAR and VARCHAR?

■ CHAR(n) stores fixed-length strings of exactly n characters — pads with spaces if shorter, consistent access time, ideal for fixed-length data (MD5 hashes, country codes). VARCHAR(n) stores variable-length strings up to n characters — uses only needed space plus 1-2 length bytes, more space-efficient for varying lengths. CHAR is slightly faster for frequently updated fields; VARCHAR for most general use cases.

Q19. What are window functions?

■ Window functions perform calculations across related rows without collapsing them (unlike GROUP BY). Syntax: FUNCTION() OVER (PARTITION BY col ORDER BY col). ROW_NUMBER(): unique sequential number. RANK(): rank with gaps for ties. DENSE_RANK(): rank without gaps. LAG(col,n): value from n rows before. LEAD(col,n): value from n rows ahead. Running SUM/AVG with OVER. Example: RANK() OVER (PARTITION BY dept ORDER BY salary DESC).

Q20. What is a CTE?

■ A CTE (Common Table Expression) is a named temporary result set defined with the WITH keyword, available for the duration of a single SQL statement. Syntax: WITH cte AS (SELECT ...) SELECT * FROM cte. Benefits: improves readability by breaking complex queries into named steps, can be referenced multiple times, supports recursion (WITH RECURSIVE) for hierarchical data like org charts. Supported in MySQL 8.0+.

Q21. What is the difference between MyISAM and InnoDB?

■ InnoDB (MySQL default since 5.5): ACID-compliant, supports transactions and foreign keys, row-level locking for concurrency, MVCC for consistent reads, crash recovery. MyISAM: no transactions, no foreign keys, table-level locking (poor concurrency for writes), faster for read-heavy workloads with no concurrent writes. InnoDB is superior for most production applications. MyISAM has very limited modern use cases.

6. Angular

◆ Core Concepts

Q1. What is Angular and how is it different from AngularJS?

■ Angular (2+) is a complete TypeScript-based rewrite of AngularJS, featuring component-based architecture, a real module system (NgModule), RxJS for reactivity, CLI tooling, and improved performance with Ivy compiler. AngularJS (1.x) used JavaScript with `$scope/controllers`. Angular has no `$scope`, supports Angular Universal (SSR), lazy loading, and better mobile support. AngularJS is in long-term support only.

Q2. What is a component in Angular?

■ A component is Angular's primary UI building block: a TypeScript class with `@Component({ selector, templateUrl, styleUrls })` decorator, an HTML template (view), and CSS styles. The selector defines the custom HTML tag. Components communicate via `@Input()` (parent to child data flow) and `@Output()` with `EventEmitter` (child to parent events). They encapsulate UI logic and follow the component lifecycle.

Q3. What is an NgModule?

■ `NgModule` (`@NgModule` decorator) organizes an Angular app into cohesive blocks. It: declares components/directives/pipes, imports other modules (`BrowserModule`, `FormsModule`, `HttpClientModule`), exports declarations for other modules, and provides services. The root `AppModule` bootstraps the app. Feature modules organize related functionality and enable lazy loading. Shared modules export reusable pieces.

Q4. What is data binding? Explain types.

■ Interpolation `{{ value }}`: one-way, component class to template. Property binding `[property]='value'`: one-way, sets a DOM property from component data. Event binding `(event)='handler()'`: one-way, template to class. Two-way binding `[(ngModel)]='property'` (banana in a box): combines property and event binding — bidirectional sync between input and class property. Requires `FormsModule`. Property binding is preferred over two-way for non-form scenarios.

Q5. What are Angular directives?

■ Directives add behavior to elements. Components are directives with templates. Structural directives modify DOM structure with `*` prefix: `*ngIf` (conditionally add/remove), `*ngFor` (repeat element), `*ngSwitch`. Attribute directives change appearance/behavior: `ngClass`, `ngStyle`, `[class.name]`. The `*` syntax is sugar that expands to `<ng-template [ngIf]='condition'>`. You can also create custom structural and attribute directives.

Q6. What is the difference between structural and attribute directives?

■ Structural directives alter the DOM layout by adding/removing elements — use `*` prefix (`*ngIf`, `*ngFor`). They access `ViewContainerRef` and `TemplateRef` to manipulate the DOM. Only one structural directive per element. Attribute directives change appearance/behavior of existing elements without adding/removing them — `ngClass`, `ngStyle`, custom directives. Multiple attribute directives can coexist on the same element.

Q7. What are Angular pipes?

■ Pipes transform values in templates: `{{ date | date:'yyyy-MM-dd' }}`, `{{ text | uppercase }}`, `{{ 1234.5 | currency:'USD' }}`, `{{ 0.75 | percent }}`, `{{ obj | json }}`, `{{ value | async }}`. `async` pipe subscribes to `Observable/Promise` and auto-manages subscription. Pure pipes (default) recalculate only on input reference change. Impure pipes recalculate every change detection cycle — use sparingly for performance.

Q8. What is the component lifecycle and its hooks?

■ In order: `ngOnChanges` (on input changes, before `init`), `ngOnInit` (after first `ngOnChanges` — setup/HTTP calls), `ngDoCheck` (custom change detection), `ngAfterContentInit` (projected content initialized), `ngAfterContentChecked`, `ngAfterViewInit` (view initialized — DOM manipulation), `ngAfterViewChecked`, `ngOnDestroy` (before destruction — unsubscribe, cleanup). `ngOnDestroy` is critical to prevent memory leaks.

Q9. What are Angular decorators?

■ `@Component`: marks component with template/selector. `@NgModule`: declares a module. `@Injectable`: marks class as DI-available. `@Input()`: marks property as parent-bound input. `@Output()`: marks `EventEmitter` for parent communication. `@Pipe`: marks a pipe class. `@HostListener`: listens to host DOM events. `@ViewChild/@ContentChild`: queries for child elements. Decorators are TypeScript metadata that Angular processes at runtime.

Q10. What is the difference between `ViewChild` and `ContentChild`?

■ `@ViewChild` accesses elements/components defined in the component's OWN template. `@ContentChild` accesses elements projected into the component via `<ng-content>` (from the parent's template). `@ViewChild` is available after `ngAfterViewInit`; `@ContentChild` after `ngAfterContentInit`. Both use a selector (class, template reference `#ref`, or directive). Use `static: true` if the reference exists unconditionally and you need it in `ngOnInit`.

◆ Services, DI, Routing & RxJS

Q11. What is dependency injection in Angular?

■ Angular's DI system provides dependencies to components/services instead of them creating their own. When a component's constructor requests a service (`constructor(private svc: MyService)`), Angular's injector creates or retrieves the instance and injects it. Promotes loose coupling, testability (easy mocking), and instance reuse. The injector hierarchy (root → module → component) determines how instances are shared.

Q12. What is the difference between `providedIn: 'root'` and module-level providers?

■ `providedIn: 'root'` (in `@Injectable`): registers in the root injector — singleton shared across the entire app, tree-shakable (removed if unused). Module-level providers (`NgModule` providers array): singleton for that module scope. Lazy-loaded modules get their own injector — services provided there are not shared with eagerly loaded modules. Component-level providers: new instance per component. `providedIn: 'root'` is the preferred default.

Q13. How does Angular routing work?

■ Angular Router maps URL paths to components. Define routes in `RouterModule.forRoot(routes)`: `{ path: 'users', component: UsersComponent }`. Place `<router-outlet>` where routed components render. Navigate programmatically with `Router.navigate()` or declaratively with `[routerLink]`. The router reads the URL, finds the matching route, and renders the component into the outlet. Child routes, guards, and lazy loading extend this.

Q14. What is lazy loading?

■ Lazy loading loads feature modules only when navigating to their routes, reducing initial bundle size and improving startup time. Implementation: `{ path: 'admin', loadChildren: () => import('./admin/admin.module').then(m => m.AdminModule) }`. Angular creates a separate JS chunk per lazy module. Combine with `PreloadingStrategy` to preload lazy modules in the background after initial load for faster navigation.

Q15. What are route guards?

■ Route guards control navigation: CanActivate: can this route be activated? (authentication). CanActivateChild: guards child routes. CanDeactivate: can you leave? (unsaved form warning). CanLoad: prevents lazy module from loading (authorization). Resolve: pre-fetches data before route activates. Implement the interface, return boolean/Observable<boolean>/UrlTree (for redirects). Add to the route's guards array.

Q16. How do you make HTTP calls in Angular?

■ Import HttpClientModule. Inject HttpClient in a service. Use: get<T>(url), post<T>(url, body), put, patch, delete — all return Observables. Subscribe in components or use async pipe in templates. Keep HTTP calls in services, not components. Handle errors with catchError() RxJS operator. Use HttpParams for query params and HttpHeaders for custom headers. HttpClient automatically serializes/deserializes JSON.

Q17. What are Angular interceptors?

■ HTTP interceptors implement HttpInterceptor and intercept all HTTP requests/responses. Common uses: add auth token to all requests, handle errors globally, show loading spinner, log requests. Register in AppModule: { provide: HTTP_INTERCEPTORS, useClass: AuthInterceptor, multi: true }. The intercept(req, next) method receives the request and a handler — call next.handle(req) to pass it along, optionally modifying the request or response.

Q18. What is the difference between Template-driven and Reactive forms?

■ Template-driven forms: defined in HTML with ngModel, simple, asynchronous, harder to unit test. Requires FormsModule. Reactive forms: structure defined in TypeScript with FormGroup/FormControl/FormArray, synchronous, easily tested, more control, better for complex dynamic forms. Requires ReactiveFormsModule. Reactive forms are generally preferred for production applications due to testability and explicit control flow.

Q19. What is RxJS and what are Observables?

■ RxJS is a library for reactive programming with observable streams. An Observable is a lazy, cancellable stream that can emit multiple values over time. Unlike Promises (single value, eager), Observables don't execute until subscribed, can be cancelled via unsubscribe(), and have a rich operator library. Angular's HttpClient, Router events, and form valueChanges all return Observables.

Q20. What is the difference between Observable and Promise?

■ Promise: single value, eager (executes on creation), not cancellable, no built-in operators. Observable: multiple values over time, lazy (executes on subscribe), cancellable (unsubscribe()), rich RxJS operators (map, filter, retry, catchError, debounceTime). Use Promises for simple one-off async operations; Observables for streams, real-time data, HTTP with retry logic, or when operators are needed.

Q21. What are switchMap, mergeMap, and combineLatest?

■ switchMap: maps outer values to inner Observables and cancels the previous inner Observable when a new outer value arrives — ideal for HTTP search (cancel previous request on new keystroke). mergeMap: subscribes to each inner Observable concurrently without cancelling — good for parallel operations. combineLatest([obs1, obs2]): emits when any source emits, combining latest values — good for combining multiple dependent data streams.

Q22. What is change detection and what is OnPush?

■ Angular's default change detection checks all components in the tree after any async event. OnPush strategy only checks a component when: its @Input reference changes, an event originates from it or its children, an Observable is used with async pipe, or changeDetectorRef.markForCheck() is called. OnPush dramatically improves performance in large apps by skipping unchanged component subtrees. Works naturally with immutable data.

7. Spring Framework

◆ Core Spring

Q1. What is the Spring Framework?

■ Spring is a comprehensive Java application framework simplifying enterprise development. It solves: tight coupling (via DI/IOC), boilerplate code (JDBC/JMS templates), cross-cutting concerns (AOP for logging/security/transactions), and complex J2EE APIs. Provides lightweight, POJO-based programming, testability, and a rich ecosystem (Spring MVC, Spring Data, Spring Security, Spring Boot).

Q2. What is the Spring IoC container?

■ The IoC container manages bean lifecycle and dependencies. It reads configuration (annotations, Java @Configuration, or XML), creates beans, injects dependencies, and destroys beans on context close. Two types: BeanFactory (basic, lazy init) and ApplicationContext (extends BeanFactory, adds AOP, event publishing, i18n, web support — the standard choice). Implementations: ClassPathXmlApplicationContext, AnnotationConfigApplicationContext.

Q3. What is Inversion of Control?

■ IoC is a design principle where control of object creation and dependency management transfers from application code to a framework/container. Instead of objects creating their own dependencies (new Service()), the container creates and provides them — inverting the traditional flow. Benefits: loose coupling, easier testing (inject mocks), and flexible configuration. Dependency Injection is the primary implementation of IoC.

Q4. What are the types of Dependency Injection in Spring?

■ Constructor Injection: dependencies via constructor — preferred, enables immutability, makes dependencies explicit. Setter Injection: via setter methods — good for optional dependencies. Field Injection (@Autowired on field): not recommended — hides dependencies, makes testing harder, prevents final fields. Spring recommends constructor injection for mandatory dependencies and setter injection for optional ones.

Q5. What is a Spring Bean and its scopes?

■ A Spring bean is an object managed by the IoC container. Scopes: singleton (default — one instance per container, shared), prototype (new instance per injection/request), request (new per HTTP request), session (new per HTTP session), application (one per ServletContext), websocket (one per WebSocket session). Singleton is most common; prototype for stateful beans that should not be shared.

Q6. What is the difference between @Component, @Service, @Repository, and @Controller?

■ All are @Component specializations enabling component scanning. @Component is the generic stereotype. @Service marks service layer classes — no extra behavior currently. @Repository marks DAO classes — enables automatic exception translation (converts JPA/JDBC exceptions to Spring's DataAccessException). @Controller marks MVC controllers. @RestController = @Controller + @ResponseBody. Use specific stereotypes for clarity and to benefit from their specific behaviors.

Q7. What is @Autowired?

■ @Autowired injects dependencies automatically by searching the application context for a bean matching the required type. Multiple type matches are resolved using @Qualifier or bean name. Can be on constructors (implicit for single-constructor classes in Spring 4.3+), fields, or setters. required=false makes the dependency optional (null if not found). Constructor injection is preferred over @Autowired on fields.

Q8. What is the difference between @Qualifier and @Primary?

■ @Primary marks a bean as the default choice when multiple beans of the same type exist — injected when @Qualifier is not used. @Qualifier('beanName') explicitly specifies which bean to inject when multiple candidates exist. @Qualifier takes precedence over @Primary when both are present. Use @Primary for the most commonly used implementation; @Qualifier for specific selections.

Q9. What is ApplicationContext vs BeanFactory?

■ BeanFactory is the basic IoC container with lazy initialization and minimal functionality. ApplicationContext extends BeanFactory and adds: eager singleton initialization, automatic BeanPostProcessor registration, event publishing, MessageSource for i18n, and ResourceLoader. Always use ApplicationContext for application code. BeanFactory is only useful in extreme memory-constrained environments.

Q10. What is the Spring Bean lifecycle?

■ Order: 1) Instantiate (constructor), 2) Inject dependencies, 3) Aware interfaces (BeanNameAware, ApplicationContextAware), 4) BeanPostProcessor.postProcessBeforeInitialization(), 5) @PostConstruct / InitializingBean.afterPropertiesSet() / init-method, 6) BeanPostProcessor.postProcessAfterInitialization() — bean is ready. On shutdown: 7) @PreDestroy / DisposableBean.destroy() / destroy-method. @PostConstruct and @PreDestroy are the preferred lifecycle callbacks.

Q11. What is the difference between @Bean and @Component?

■ @Component is placed on the class itself, detected via component scanning — the class must be directly instantiable by Spring. @Bean is placed on a method inside a @Configuration class — you write the instantiation code yourself. Use @Bean for: configuring third-party classes you cannot annotate, complex instantiation logic, or when configuring specific parameters. @Bean gives full creation control; @Component is more concise for your own classes.

◆ AOP, MVC & Data

Q12. What is AOP?

■ AOP (Aspect-Oriented Programming) separates cross-cutting concerns (logging, security, transactions, caching) from business logic. Instead of scattering them throughout code, they are modularized into Aspects. Spring AOP uses JDK dynamic proxies or CGLIB to intercept method calls and apply aspect logic. Benefits: cleaner code, no duplication, easy to add/remove concerns without changing business logic.

Q13. What are the key AOP concepts?

■ Aspect: module encapsulating cross-cutting concern (@Aspect class). Advice: action taken at a join point (actual code that runs). Join Point: execution point — Spring AOP only supports method execution. Pointcut: expression defining which join points trigger advice (execution(* com.example.service.*(..))). Target Object: the object being advised. Weaving: linking aspects with objects at runtime via proxies.

Q14. What are the types of advice?

■ @Before: runs before the method. @After: runs after (success or exception). @AfterReturning: runs after successful return, can access return value. @AfterThrowing: runs after exception is thrown. @Around: most powerful — wraps the method, controls execution via ProceedingJoinPoint.proceed(). @Around can prevent execution, modify args/return value, handle exceptions. Use @Around for transactions/caching, @Before/@After for logging.

Q15. What is Spring MVC and how does request flow work?

■ Spring MVC implements the Model-View-Controller pattern. Request flow: 1) Request to DispatcherServlet. 2) HandlerMapping finds the @Controller method. 3) HandlerAdapter invokes it. 4) Controller processes request (calls service/DAO), returns ModelAndView or data. 5) ViewResolver resolves view name to template (Thymeleaf, JSP). 6) View renders response. For REST APIs, @ResponseBody bypasses view resolution and writes directly to the response body.

Q16. What is DispatcherServlet?

■ DispatcherServlet is Spring MVC's front controller — a single servlet handling all incoming HTTP requests. It coordinates: HandlerMapping (find handler), HandlerAdapter (invoke handler), ViewResolver (resolve view), HandlerExceptionResolver (handle exceptions). All Spring MVC configuration (interceptors, message converters) flows through it. Auto-configured by Spring Boot; manually configured via web.xml or WebApplicationInitializer in traditional Spring MVC.

Q17. What is the difference between @RequestParam and @PathVariable?

■ @PathVariable extracts values from URI path segments: @GetMapping('/users/{id}') + @PathVariable Long id → maps /users/42. @RequestParam extracts query string parameters: @GetMapping('/users') + @RequestParam String name → maps /users?name=John. @RequestParam supports defaultValue and required=false. Use @PathVariable for resource identifiers in REST URLs; @RequestParam for optional filters/pagination/search terms.

Q18. What is @RestController vs @Controller?

■ @Controller returns view names resolved by a ViewResolver — for server-side rendered web pages (JSP, Thymeleaf). @RestController = @Controller + @ResponseBody on every method — serializes return values directly to HTTP response body (JSON/XML via Jackson). Use @RestController for REST APIs. Use @Controller when mixing view responses and API responses (apply @ResponseBody selectively per method).

Q19. What is Spring Data JPA?

■ Spring Data JPA simplifies JPA-based data access by generating repository implementations at runtime. Define an interface extending JpaRepository<Entity, ID> — you get 50+ CRUD, pagination, and sorting methods automatically. Custom queries via method naming conventions (findByLastNameAndAge), @Query (JPQL/SQL), or Specifications. Eliminates boilerplate DAO code. Also supports MongoDB, Redis, and other stores.

Q20. What is JpaRepository?

■ JpaRepository<T, ID> extends PagingAndSortingRepository extends CrudRepository. Key methods: save(), saveAll(), findById() (returns Optional<T>), findAll(), findAll(Pageable), findAll(Sort), delete(), deleteById(), count(), existsById(). JPA-specific: flush(), saveAndFlush(), deleteInBatch(). Custom finders by naming: findByEmail(), findByAgeGreaterThan(). Supports Pageable for pagination and Sort for ordering without additional code.

Q21. What is @Transactional?

■ @Transactional manages transaction boundaries declaratively. Spring wraps the method in a transaction (begins before, commits after, rolls back on RuntimeException by default). Key attributes: propagation (REQUIRED — join existing or create, REQUIRES_NEW — always create), isolation (READ_COMMITTED, etc.), readOnly (performance hint), rollbackFor (specify exception types that trigger rollback), timeout. Apply on service layer methods.

8. Spring Boot

◆ Core & Configuration

Q1. What is Spring Boot and how is it different from Spring?

■ Spring Boot removes configuration burden with: auto-configuration (auto-configures beans based on classpath and properties), embedded servers (run as JAR without WAR deployment), starter dependencies (curated dependency sets), production features (Actuator), and opinionated defaults. Plain Spring requires manual `DataSource`, `DispatcherServlet`, `EntityManagerFactory` configuration. Spring Boot configures these automatically so developers focus on business logic.

Q2. What is auto-configuration?

■ Auto-configuration automatically configures Spring beans based on classpath contents (which JARs present), existing beans, and property settings. Enabled by `@EnableAutoConfiguration` (part of `@SpringBootApplication`). Spring Boot scans for `AutoConfiguration` classes in `spring.factories/AutoConfiguration.imports`. Each class uses `@Conditional` annotations (`@ConditionalOnClass`, `@ConditionalOnMissingBean`) to activate only when appropriate. Disable with `spring.autoconfigure.exclude`.

Q3. What is `@SpringBootApplication`?

■ `@SpringBootApplication` combines three annotations: `@Configuration` (source of `@Bean` definitions), `@EnableAutoConfiguration` (enables auto-configuration), and `@ComponentScan` (scans current package and sub-packages for components). Place on the main class. Scanning starts from the annotated class's package — put it in the root package to scan all sub-packages. Exclude auto-configurations: `@SpringBootApplication(exclude={DataSourceAutoConfiguration.class})`.

Q4. What are Spring Boot starters?

■ Starters are curated dependency sets providing everything for a specific feature. Examples: `spring-boot-starter-web` (Tomcat, Spring MVC, Jackson), `spring-boot-starter-data-jpa` (Spring Data JPA, Hibernate, JDBC), `spring-boot-starter-security`, `spring-boot-starter-test` (JUnit 5, Mockito, AssertJ), `spring-boot-starter-actuator`. Each starter transitively includes compatible library versions managed by the `spring-boot-dependencies` BOM. No more manual version management.

Q5. What is `application.properties` vs `application.yml`?

■ Both configure Spring Boot. `application.properties` uses flat `key=value`: `server.port=8080`. `application.yml` uses YAML hierarchy with indentation — more readable for nested config and supports lists natively. Both support profile-specific files: `application-dev.properties`, `application-prod.yml`, activated by `spring.profiles.active`. Configuration override order: command-line args > environment variables > profile-specific > default properties file.

Q6. What are Spring Boot profiles?

■ Profiles enable environment-specific configuration (dev, test, prod). Activate with: `spring.profiles.active=prod` (properties), `--spring.profiles.active=prod` (command line), `SPRING_PROFILES_ACTIVE` environment variable. Profile-specific files override base properties. Beans conditionally registered with `@Profile('prod')`. Multiple profiles can be active simultaneously. `@ActiveProfiles` activates profiles in tests. Default profile applies when none is specified.

Q7. How do you change the default port?

■ Set `server.port=9090` in `application.properties/yml`. For random port: `server.port=0`. Via command line: `java -jar app.jar --server.port=9090`. Via environment variable: `SERVER_PORT=9090`. In tests: `@SpringBootTest(webEnvironment=WebEnvironment.RANDOM_PORT)` and inject with `@LocalServerPort`. Programmatically via `WebServerFactoryCustomizer` bean.

Q8. What is Spring Boot DevTools?

■ `spring-boot-devtools` improves developer experience: automatic restart when classpath files change (uses two classloaders — much faster than full restart), `LiveReload` support (auto-refreshes browser), and disabled caching for templates and static resources. Automatically disabled in production (fully packaged JAR). Add as optional Maven/Gradle dependency to exclude from production builds.

Q9. What is Spring Boot Actuator?

■ Actuator provides production-ready monitoring and management. Add `spring-boot-starter-actuator`. Common endpoints: `/actuator/health` (app health), `/actuator/info`, `/actuator/metrics` (JVM/HTTP/DB metrics), `/actuator/env` (environment properties), `/actuator/beans` (all beans), `/actuator/mappings` (all request mappings), `/actuator/loggers` (view/change log levels at runtime), `/actuator/threaddump`, `/actuator/heapdump`. Secured by default — configure `management.endpoints.web.exposure.include`.

◆ REST, Security & Testing

Q10. How do you create a REST API with Spring Boot?

■ 1) Add `spring-boot-starter-web`. 2) Create `@RestController` class. 3) Map endpoints with `@GetMapping`, `@PostMapping`, `@PutMapping`, `@DeleteMapping`, `@PatchMapping`. 4) Use `@PathVariable`, `@RequestParam`, `@RequestBody`. 5) Create service and repository layers. 6) Return `ResponseEntity<T>` for full HTTP control. Follow REST conventions: plural resource names, proper HTTP methods, appropriate status codes (200 OK, 201 Created, 204 No Content, 404 Not Found).

Q11. What is ResponseEntity?

■ `ResponseEntity<T>` gives full control over HTTP response: status code, headers, and body. Examples: `ResponseEntity.ok(body)` → 200 with body. `ResponseEntity.created(location).body(obj)` → 201 with Location header. `ResponseEntity.noContent().build()` → 204. `ResponseEntity.notFound().build()` → 404. `ResponseEntity.status(HttpStatus.FORBIDDEN).body(error)` → 403. Use it when you need specific status codes or custom headers beyond the default 200.

Q12. How does Spring Boot handle exceptions globally?

■ Use `@ControllerAdvice` with `@ExceptionHandler` methods to handle exceptions across all controllers centrally. `@RestControllerAdvice` = `@ControllerAdvice` + `@ResponseBody`. Example: `@ExceptionHandler(ResourceNotFoundException.class)` returns `ResponseEntity` with 404. Spring 6 supports `ProblemDetail` (RFC 7807 standard error format). Extend `ResponseEntityExceptionHandler` to customize Spring MVC's default exception handling.

Q13. What is @ControllerAdvice and @ExceptionHandler?

■ `@ControllerAdvice` is applied globally to all controllers, providing centralized exception handling, data binding, and model attributes. `@ExceptionHandler` annotates methods to handle specific exception types — called when the exception propagates from any controller. Can specify multiple exception types. Combined with `@ResponseStatus` or `ResponseEntity` return type to control HTTP status. Centralizes error handling, eliminating try-catch in every controller.

Q14. How do you connect Spring Boot to a database?

■ 1) Add `spring-boot-starter-data-jpa` and database driver (`mysql-connector-j`). 2) Configure in `application.properties`: `spring.datasource.url`, `.username`, `.password`; `spring.jpa.hibernate.ddl-auto=update`. 3) Create `@Entity` classes with `@Id`. 4) Create `JpaRepository` interfaces. Spring Boot auto-configures `DataSource`, `EntityManagerFactory`, and `TransactionManager` based on the classpath and properties. No explicit bean configuration needed.

Q15. What is Spring Security?

■ Spring Security (`spring-boot-starter-security`) provides comprehensive authentication and authorization. Auto-configuration secures all endpoints with HTTP Basic auth and generates a random password at startup. Customize via `SecurityFilterChain` bean: configure public/protected paths, authentication (form login, JWT, OAuth2), and authorization rules (`hasRole`, `isAuthenticated`). Core concepts: Authentication (who are you?), Authorization (what can you do?), `SecurityFilterChain`.

Q16. What is JWT with Spring Boot?

■ JWT (JSON Web Token) is a stateless authentication token: Header (algorithm) + Payload (claims: `userId`, `roles`, `expiry`) + Signature (prevents tampering). Flow: user logs in → server validates credentials → returns JWT → client includes JWT in Authorization: Bearer <token> header → server validates signature and expiry on each request — no session storage needed. Use the `jjwt` library. Implement `JwtAuthenticationFilter` extending `OncePerRequestFilter`.

Q17. What is @SpringBootTest?

■ `@SpringBootTest` loads the full Spring application context for integration testing. With `webEnvironment=RANDOM_PORT` or `DEFINED_PORT`, starts a real embedded server. `MOCK` (default) mocks the servlet environment. Use `TestRestTemplate` or `MockMvc` for HTTP testing. Slower than unit tests but tests real integration. Combine with `@Transactional` to rollback DB changes after each test. Use for integration tests; prefer `@WebMvcTest` for controller-only tests.

Q18. What is the difference between @WebMvcTest and @SpringBootTest?

■ `@WebMvcTest` loads only the web layer (controllers, `@ControllerAdvice`, filters) — services and repositories not loaded, must be `@MockBean`. Fast, isolated controller tests. Use `MockMvc`. `@SpringBootTest` loads the full application context — slower but tests true integration. Rule: `@WebMvcTest` for controller unit tests, `@SpringBootTest` for integration tests requiring the full context (real DB queries, complete request-response cycle).

Q19. What is MockMvc?

■ `MockMvc` provides a fluent API to test Spring MVC controllers without starting an HTTP server. Example: `mockMvc.perform(get('/api/users').accept(MediaType.APPLICATION_JSON)).andExpect(status().isOk()).andExpect(jsonPath('$[0].name').value('John')).andDo(print())`. Works with `@WebMvcTest` (preferred) or `@SpringBootTest`. `jsonPath` uses `Jayway JsonPath` for JSON assertions. Supports all HTTP methods with headers, body, and query params.

Q20. How do you handle CORS in Spring Boot?

■ Options: 1) `@CrossOrigin` annotation on `@RestController` or specific methods. 2) Global via `WebMvcConfigurer` bean: `registry.addMapping('/**').allowedOrigins('http://localhost:4200')`. 3) Via Spring Security `CorsConfigurationSource` bean — required when Spring Security is present. Configure allowed origins, methods, headers, credentials, and max age. Never use wildcard (*) origins with credentials in production.

Q21. What is Swagger/OpenAPI in Spring Boot?

■ Swagger/OpenAPI provides interactive API documentation. Add `springdoc-openapi-starter-webmvc-ui`. Access Swagger UI at `/swagger-ui.html` and spec at `/v3/api-docs`. Annotate: `@OpenAPIDefinition` for app info, `@Tag` on controllers, `@Operation` on methods, `@Parameter` on params, `@ApiResponse` for responses. Secure the endpoint in production. Use the spec to generate client SDKs and test APIs interactively.

9. Maven

◆ Maven Essentials

Q1. What is Maven?

■ Maven is a build automation and project management tool for Java. It standardizes the build process, manages dependencies (automatic download from repositories), enforces project structure (src/main/java, src/test/java), and provides a build lifecycle (compile, test, package, install, deploy). Solves classpath management, transitive dependencies, and repeatable builds. Alternative: Gradle (more flexible, Kotlin DSL, incremental builds).

Q2. What is a POM file?

■ POM (Project Object Model) is Maven's project descriptor — pom.xml. Key elements: groupId (organization: com.example), artifactId (project name), version, packaging (jar/war/pom), parent, properties, dependencies (with groupId, artifactId, version, scope), dependencyManagement, build/plugins, repositories, distributionManagement. Maven uses these to orchestrate the entire build lifecycle.

Q3. What are Maven repositories?

■ Local repository (~/.m2/repository): Maven caches downloaded artifacts here — checked first. Central repository (repo.maven.apache.org): Maven's default public repository with most open-source libraries — no configuration needed. Remote repository: custom private repositories (Nexus, Artifactory) for proprietary artifacts. Resolution order: local → configured remotes → central. SNAPSHOT versions are always re-checked from remote.

Q4. What is Maven's build lifecycle?

■ Default lifecycle: validate → compile → test → package → verify → install → deploy. Each phase runs all preceding phases. Common commands: mvn compile, mvn test, mvn package (creates JAR/WAR in target/), mvn install (copies to local repo), mvn deploy (uploads to remote repo), mvn clean package (clean old target then package). Clean lifecycle: pre-clean → clean → post-clean. Site lifecycle generates project documentation.

Q5. What is the difference between dependency scopes?

■ compile (default): on all classpaths, included in final artifact. test: only for testing, excluded from final artifact (JUnit, Mockito). provided: available at compile/test time but provided by runtime environment (servlet-api provided by app server). runtime: not needed for compilation but needed at runtime (JDBC driver). system: like provided but you specify path — avoid. import: in dependencyManagement to import a BOM.

Q6. What is a Maven parent POM?

■ A parent POM is shared configuration from which child projects inherit: dependencies, dependency versions (from dependencyManagement), plugin configurations, and properties. Specified in child pom.xml with <parent>. Spring Boot parent (spring-boot-starter-parent) provides Java version, encoding, plugin versions, and BOM for all starters. Company parent POMs standardize configuration across projects. The parent itself has packaging=pom.

Q7. What is a Maven multi-module project?

■ A multi-module project has a parent POM (packaging=pom) with <modules> listing sub-modules. Each sub-module has pom.xml with <parent> reference. Running mvn install from parent builds all modules in dependency order. Benefits: shared configuration, inter-module dependencies, single build command. Common in microservices and large monoliths: parent → core, api, web, service modules. Parent manages versions for consistency.

Q8. What is a SNAPSHOT version?

■ SNAPSHOT (1.0-SNAPSHOT) represents an unstable in-progress version. Maven always re-downloads SNAPSHOTs from remote repositories (checking for updates based on updatePolicy — daily by default), while release versions are cached locally after first download. Use SNAPSHOT during development. Remove SNAPSHOT before final release (1.0). SNAPSHOT artifacts should not be used in production builds.

Q9. How do you skip tests in Maven?

■ Skip execution but compile tests: `mvn package -DskipTests`. Skip compilation AND execution: `mvn package -Dmaven.test.skip=true`. In `pom.xml`: `<properties><maven.test.skip>true</maven.test.skip></properties>`. Run specific test: `mvn test -Dtest=MyTest`. Run specific method: `mvn test -Dtest=MyTest#myMethod`. Never skip tests in regular CI/CD builds. `-DskipTests` still catches compilation errors; `-Dmaven.test.skip` skips compilation too.

Q10. What is the difference between `mvn package` and `mvn install`?

■ `mvn package` compiles, tests, and creates the artifact in `target/` — local only. `mvn install` does everything `package` does PLUS copies the artifact to the local Maven repository (`~/.m2/`), making it available as a dependency for other local projects. `mvn deploy` uploads to a remote repository for team sharing. In multi-module builds, `install` ensures sibling module JARs are found in local repo by dependent modules.

Q11. What is dependencyManagement?

■ `<dependencyManagement>` declares dependency versions and scopes without actually adding them to the project's classpath. Child modules or the `<dependencies>` section can then reference those dependencies without specifying version — they inherit from `dependencyManagement`. Used in parent POMs and BOM (Bill of Materials) files imported with `scope=import`. Centralizes version management, preventing conflicts across modules.

Q12. How do you resolve dependency conflicts in Maven?

■ Maven's nearest-wins rule: the version closest to your project in the dependency tree wins. `mvn dependency:tree` shows the full dependency tree. To force a specific version: explicitly add the dependency to your `pom.xml` (overrides transitive). Use `<exclusion>` to exclude a transitive dependency: `<exclusions><exclusion><groupId>...</exclusion></exclusions>`. Use `dependencyManagement` in the parent to enforce versions across all modules. `mvn dependency:analyze` finds unused/undeclared deps.

Q13. What is the maven-surefire-plugin?

■ `maven-surefire-plugin` runs unit tests during the test phase. Discovers tests by convention: `Test*.java`, `*Test.java`, `*Tests.java`, `*TestCase.java`. Supports JUnit 4/5 and TestNG. Configure in `pom.xml` for: includes/excludes patterns, system properties, parallel execution (`forkCount`, parallel settings), JVM args. Reports in `target/surefire-reports/`. For integration tests, use `maven-failsafe-plugin` (`*IT.java`, runs during verify phase).

Q14. How do you create an executable JAR?

■ `maven-assembly-plugin` with `jar-with-dependencies` creates a fat JAR. `maven-shade-plugin` for shading (relocates classes to avoid classpath conflicts). For Spring Boot: `spring-boot-maven-plugin` automatically repackages the JAR to be executable with embedded server. Run with: `java -jar myapp.jar`. `MANIFEST.MF` specifies the Main-Class. The plugin is included automatically when using `spring-boot-starter-parent`.

Q15. What is the Spring Boot Maven plugin?

■ spring-boot-maven-plugin integrates Spring Boot with Maven. Key goals: spring-boot:run (run without packaging), spring-boot:repackage (create executable fat JAR — runs automatically during package phase), spring-boot:build-image (build OCI image with Buildpacks). Automatically included with spring-boot-starter-parent. Essential for creating deployable Spring Boot artifacts. Main class is auto-detected or configurable.

10. Git

◆ Core Concepts & Commands

Q1. What is Git and how is it different from SVN?

■ Git is a distributed VCS — every developer has a full repository copy with complete history, enabling offline work, fast operations, and no single point of failure. SVN is centralized — a single server holds the repository; clients check out working copies. Git operations like commit, log, and diff are local. Git supports powerful branching/merging; SVN branching is expensive (full copy). Git is the industry standard; SVN is largely legacy.

Q2. What are the three states in Git?

■ Working directory: where you edit files — changes are untracked or modified. Staging area (index): files marked for the next commit with `git add` — a prepared snapshot. Git repository (.git): the database of all committed snapshots. Workflow: edit in working directory → stage with `git add` → commit with `git commit`. `git status` shows file states. `git diff` shows working directory vs staging; `git diff --staged` shows staging vs last commit.

Q3. What is the difference between git fetch and git pull?

■ `git fetch` downloads remote changes but does NOT merge them — safe, non-destructive. Local branches unchanged; remote-tracking branches (origin/main) updated. `git pull` = `git fetch` + `git merge` (or rebase with `--rebase` flag). `git pull` is convenient but can create unexpected merge commits. Best practice: `git fetch` first, review with `git log origin/main`, then merge or rebase intentionally.

Q4. What is git stash?

■ `git stash` temporarily saves uncommitted changes and reverts working directory to a clean state — useful when switching branches without committing incomplete work. Commands: `git stash` (save), `git stash list` (view all), `git stash pop` (apply and remove latest), `git stash apply` (apply without removing), `git stash drop` (remove specific), `git stash branch <branch>` (create branch from stash). Entries identified as `stash@{0}`, `stash@{1}`.

Q5. What is git reset?

■ `git reset` moves the branch pointer to a different commit. `--soft`: moves HEAD only — all changes remain staged (commits undone but changes preserved). `--mixed` (default): moves HEAD and clears staging area — changes remain in working directory as unstaged. `--hard`: moves HEAD, clears staging, AND reverts working directory — all changes permanently lost. Use `--soft` to combine commits; `--mixed` to unstage; `--hard` carefully when you are certain about discarding changes.

Q6. What is the difference between git reset and git revert?

■ `git reset` rewrites history by moving the branch pointer back — dangerous on shared branches as it removes commits others may have. `git revert` creates a NEW commit that undoes changes of a specific commit — history is preserved, safe for shared/public branches. Use `reset` for local unpublished commits to clean up before pushing. Use `revert` for shared commits on remote branches. `git revert --no-commit` applies changes without immediately committing.

Q7. What is git cherry-pick?

■ `git cherry-pick` applies changes from a specific commit onto the current branch, creating a new commit with a different SHA. Useful for: applying a bug fix from one branch to another without merging the entire branch. Example: `git cherry-pick abc1234`. Range: `git cherry-pick abc..xyz`. `--no-commit` applies changes without committing first. Conflicts must be resolved manually if they occur.

Q8. What is git reflog?

■ git reflog records every movement of HEAD — resets, checkouts, rebases — even if commits are no longer referenced by any branch. Your safety net for recovering 'lost' commits after hard resets or accidental deletions. Entries expire after 90 days. Use git reflog to find a lost commit's SHA, then git checkout <sha> or git branch <name> <sha> to recover it. Think of it as Git's undo history for HEAD movements.

Q9. What is HEAD in Git?

■ HEAD is a pointer to the currently checked-out commit or branch. Normally HEAD points to a branch (like main), and the branch points to the latest commit — HEAD is 'attached'. When you checkout a specific commit (not a branch), HEAD points directly to that commit — 'detached HEAD'. HEAD~1 or HEAD^ refers to one commit before HEAD; HEAD~2 two commits back.

Q10. What is a detached HEAD state?

■ Detached HEAD occurs when HEAD points to a specific commit rather than a branch — happens when you checkout a commit hash, tag, or remote branch. You can browse and even make commits, but new commits are not on any branch and will eventually be garbage collected. To preserve work: immediately create a branch with git checkout -b new-branch. Common triggers: git checkout <sha>, git checkout v1.0.0.

◆ Branching, Merging & Workflows

Q11. What is the difference between git merge and git rebase?

■ git merge combines branches by creating a new merge commit with two parents, preserving the original branch history — non-destructive. git rebase moves/re-applies commits from one branch on top of another by rewriting commit history (new SHAs) — creates linear history. Golden rule: never rebase public/shared branches as it rewrites history others depend on. Merge preserves true history; rebase creates cleaner linear history.

Q12. When would you use rebase vs merge?

■ Use rebase for: updating a feature branch with latest main (git rebase main — cleaner than merge commit), cleaning up local commits before pushing (interactive rebase git rebase -i), keeping feature branches linear. Use merge for: integrating completed features into main (preserves branch history), shared/public branches (history must not be rewritten), when you want explicit merge commit showing integration point. Team workflow should define the standard.

Q13. What is a fast-forward merge?

■ A fast-forward merge occurs when the current branch has not diverged from the incoming branch — all commits are linear ancestors. Git simply moves the branch pointer forward without creating a merge commit. Use --no-ff to always create a merge commit even when fast-forward is possible, preserving feature branch history. Most PR merges avoid fast-forward to maintain visible merge commits in the main branch history.

Q14. How do you resolve a merge conflict?

■ Conflicts occur when two branches modify the same file region. Git marks conflicts with <<<<<< HEAD, =====, >>>>>> branch. Resolution: 1) Open conflicted files (git status shows them). 2) Edit each file keeping desired changes (remove conflict markers). 3) Stage with git add. 4) Complete with git commit. Use git mergetool for visual diff. git merge --abort cancels and restores pre-merge state if needed.

Q15. What is a Git tag?

■ Tags mark specific commits as important, typically releases (v1.0.0). Lightweight tag: git tag v1.0 — just a pointer. Annotated tag: git tag -a v1.0 -m 'Release 1.0' — stores tagger info, date, message — recommended for releases. List: git tag. Push: git push origin v1.0 or git push --tags. Delete locally: git tag -d v1.0; remotely: git push --delete origin v1.0. Tags are not pushed by default with git push.

Q16. What are common Git workflows?

■ Git Flow: main (production) + develop + feature/*, release/*, hotfix/* branches — structured, good for scheduled releases. GitHub Flow: main (always deployable) + feature branches → PR → merge → deploy — simple CI/CD friendly, good for continuous delivery. Trunk-Based Development: short-lived branches (<1 day), frequent merges to main, feature flags for incomplete work — best for high-velocity teams with strong CI. GitLab Flow adds environment branches between these extremes.

Q17. What is .gitignore?

■ .gitignore specifies files Git should not track. Patterns: *.log, target/ (directory), !important.log (exception), /build (only root), **/*.class (all .class files). Only applies to untracked files — already tracked files must be removed with `git rm --cached`. Repository-level, global (`~/.gitignore_global` for .DS_Store, .idea/), and directory-level files are supported. GitHub provides language-specific .gitignore templates. Always create before first commit.

Q18. What is `git remote add origin`?

■ `git remote add origin <url>` adds a reference to a remote repository named 'origin' to your local repository. 'origin' is the conventional name for the primary remote. After adding, `git push -u origin main` sets the upstream tracking relationship. `git remote -v` lists all remotes. Typical workflow: `git init` → `git add .` → `git commit` → `git remote add origin <url>` → `git push -u origin main`. You can have multiple remotes (origin, upstream, backup).

11. GitHub

◆ GitHub Features & Collaboration

Q1. What is GitHub?

■ GitHub is a cloud-based Git repository hosting and collaborative development platform. Key features: repository hosting (public/private), pull requests (code review + merge), Issues (bug tracking/project management), GitHub Actions (CI/CD), GitHub Pages (static site hosting), Dependabot (automated dependency updates), GitHub Projects (kanban boards), Packages (artifact registry), Copilot (AI coding assistant), and code/secret scanning.

Q2. What is a pull request?

■ A PR proposes changes from one branch (or fork) to another. It shows a diff, allows collaborators to review code, leave line-level and general comments, approve or request changes, and run CI checks automatically. Required approvals and passing checks gate the merge button. PRs document why changes were made, enable knowledge sharing, catch bugs before merging, and maintain code quality. They are the core of collaborative GitHub development.

Q3. What is the difference between fork and clone?

■ Fork creates a server-side copy under your GitHub account — used to contribute to projects you don't have write access to. Make changes in your fork, then create a PR to the original (upstream) repo. Clone creates a local copy of any repository on your machine: `git clone URL`. Workflow for open-source: fork on GitHub → clone your fork locally → make changes → push to your fork → create PR to upstream. Configure upstream remote to sync with original.

Q4. What are GitHub Issues?

■ Issues are GitHub's built-in task tracker for bugs, features, and discussions. Features: labels (bug, enhancement, help-wanted), milestones (group for releases), assignees, comments, cross-references (#issue-number), auto-closing via commit message (Fixes #123), and issue templates for standardized formats. Issues integrate with Projects boards. They serve as the primary communication and planning tool for GitHub-based projects.

Q5. What are branch protection rules?

■ Branch protection rules enforce policies on branches (main, develop): require PR reviews (N approvals), require status checks to pass (CI must pass), require linear history (enforce squash/rebase), require signed commits, restrict direct push, require up-to-date branches, prevent force pushes and deletion. Set in Settings → Branches. Essential for maintaining main branch stability and enforcing code review in team projects.

Q6. What is the difference between squash merge, rebase merge, and merge commit?

■ Merge commit (Create a merge commit): preserves all individual commits + creates a merge commit — full history but noisy. Squash and merge: combines all PR commits into ONE commit on the base branch — clean linear history, loses individual commit granularity. Rebase and merge: replays PR commits on top of base, linear history with all commits but SHA changes. Teams should pick one strategy. Squash keeps main history clean; merge commit preserves full feature history.

Q7. What are GitHub Actions?

■ GitHub Actions is a CI/CD platform integrated directly into GitHub. Workflows are YAML files in `.github/workflows/` triggered by events: push, pull_request, schedule (cron), workflow_dispatch (manual). Workflows have jobs → steps. Steps run shell commands or use Actions (reusable steps from Marketplace). Jobs run on GitHub-hosted runners (ubuntu-latest, windows-latest, macos-latest) or self-hosted. Free for public repositories.

Q8. What is a GitHub workflow file?

■ A YAML file in `.github/workflows/` defining automated CI/CD. Structure: name (workflow name), on (trigger events), env (environment variables), jobs (parallel job definitions with: runs-on, steps). Steps use: uses (action reference like actions/checkout@v4) or run (shell command). Example: on: push: branches: [main] → jobs: build: runs-on: ubuntu-latest → steps: uses: actions/checkout@v4, run: mvn test. Version-controlled automation-as-code.

Q9. What are GitHub secrets?

■ Secrets are encrypted environment variables stored in GitHub (Settings → Secrets → Actions), injected into workflows at runtime. Never displayed in logs after creation. Access: `${{ secrets.API_KEY }}`. Types: repository (single repo), organization (shared across repos with policy), environment (tied to deployment environments with approval gates). Never hardcode credentials in workflow files. Rotate secrets regularly. `secrets.GITHUB_TOKEN` is automatically provided.

Q10. What is a CODEOWNERS file?

■ CODEOWNERS (`.github/CODEOWNERS`) automatically assigns reviewers to PRs based on changed files. Syntax: `/docs/ @alice` (Alice reviews docs changes). `*.java @backend-team` (backend team reviews Java files). Combined with branch protection 'require review from code owners', ensures subject matter experts review relevant changes. Last matching pattern wins. Owners can be individual users or teams. Reduces overhead of manually assigning reviewers.

Q11. What is Dependabot?

■ Dependabot automates dependency management. It scans repositories for outdated or vulnerable dependencies and creates PRs to update them. Features: security alerts (notifies about CVE vulnerabilities), security updates (auto-PR to fix vulnerabilities), version updates (regular update PRs on a schedule). Configure with `.github/dependabot.yml` specifying package ecosystem (maven, npm, docker), directory, and update schedule. Reduces security risk from unpatched dependencies.

Q12. What are GitHub webhooks?

■ Webhooks send HTTP POST requests to a configured URL when GitHub events occur (push, PR opened, release created). The JSON payload contains event details. Used to: trigger external CI/CD systems, update project management tools, send notifications, sync with external services. Configure in Settings → Webhooks. Specify Payload URL, content type, secret (for HMAC signature verification), and which events to trigger on. Verify X-Hub-Signature header in your receiver.

Q13. How do you resolve a merge conflict in a pull request?

■ Via GitHub web editor: click 'Resolve conflicts' button → edit conflicted files in browser → mark as resolved → commit merge. Via command line: checkout the PR branch → git fetch origin → git merge origin/main → resolve conflicts → git add resolved files → git commit → git push → PR updates automatically. For complex conflicts, command-line resolution is preferred. After resolution, PR shows 'No conflicts' and can proceed to merge.

12. Selenium

◆ Core WebDriver

Q1. What is Selenium and its components?

■ Selenium is an open-source web browser automation framework. Components: Selenium IDE (browser extension for record-and-playback, quick prototyping), Selenium WebDriver (programmatic browser control API — the core component used in automation), Selenium Grid (distributed parallel test execution across multiple machines/browsers). Selenium 4 uses the W3C WebDriver protocol, enabling BiDi (bidirectional) browser communication.

Q2. What are the different locator strategies?

■ 8 locator strategies: By.id (fastest — unique ID), By.name, By.className (only for unique classes), By.tagName, By.linkText (exact anchor text), By.partialLinkText, By.cssSelector (fast, flexible — recommended), By.xpath (most powerful — can traverse DOM, find by text, use conditions). Priority for maintainability: id > name > cssSelector > xpath. Avoid locators based on auto-generated IDs or absolute XPaths.

Q3. What is the difference between absolute and relative XPath?

■ Absolute XPath starts from root (/html/body/div/input) — traverses entire DOM, very fragile, breaks if any ancestor changes. Relative XPath starts from any node (//input[@id='user']) — flexible and maintainable. Relative XPath supports: attributes (@attr='value'), text content (text()='Submit'), positional predicates ([1]), axes (parent::, ancestor::, following-sibling::), and combined conditions ([@type='text' and @name='email']). Always prefer relative XPath.

Q4. What is the difference between implicit, explicit, and fluent wait?

■ Implicit wait: global setting — waits up to N seconds for any element before throwing NoSuchElementException: `driver.manage().timeouts().implicitlyWait(Duration.ofSeconds(10))`. Explicit wait: specific condition on a specific element — `WebDriverWait wait = new WebDriverWait(driver, Duration.ofSeconds(10)); wait.until(ExpectedConditions.elementToBeClickable(element))`. Fluent wait: customizable polling interval and ignored exceptions. Prefer explicit wait; never mix implicit and explicit (can double wait times).

Q5. What is the difference between findElement and findElements?

■ `findElement(By)` returns the first matching WebElement — throws NoSuchElementException immediately if not found. `findElements(By)` returns a List<WebElement> of all matches — returns empty list (no exception) if none found. Use `findElements` to check existence without try-catch: `driver.findElements(By.id('error')).isEmpty()`. Also useful for iterating: table rows, menu items, search results. `findElements` returns all matches; `findElement` only the first.

Q6. How do you handle dropdowns using the Select class?

■ For native HTML <select> elements: `Select dropdown = new Select(driver.findElement(By.id('country')))`. Methods: `selectByVisibleText('India')`, `selectByValue('IN')`, `selectByIndex(2)`. Multi-select: `selectAll()`, `deselectAll()`. Check `isMultiple()` for multi-selects. `getOptions()` returns all options. `getFirstSelectedOption()` returns current selection. For custom dropdowns (not native <select>): click the trigger, wait for options, click the option element directly.

Q7. How do you handle alerts?

■ For native browser alerts/confirm/prompts: `Alert alert = driver.switchTo().alert()`. Methods: `getText()` (get message), `accept()` (click OK), `dismiss()` (click Cancel), `sendKeys('text')` (for prompts). Always wait first: `ExpectedConditions.alertIsPresent()`. For authentication dialogs: include credentials in URL (`http://user:pass@site.com`). For web-based modal dialogs (HTML elements): treat as regular elements with `findElement`.

Q8. How do you handle multiple windows or tabs?

■ Get all handles: `Set<String> handles = driver.getWindowHandles()`. Save original: `String original = driver.getWindowHandle()`. Switch to new window: `driver.switchTo().window(handleString)`. In Selenium 4: `driver.switchTo().newWindow(WindowType.TAB)`. Switch back: `driver.switchTo().window(original)`. Close current window: `driver.close()` (then switch back to original — do NOT call quit). Find target window by title: `for(String h: handles) { driver.switchTo().window(h); if(driver.getTitle().contains('target')) break; }`

Q9. What is the difference between `driver.close()` and `driver.quit()`?

■ `driver.close()` closes only the currently focused window/tab — other windows remain open. `driver.quit()` terminates the entire WebDriver session, closes ALL windows, and kills the browser and driver processes (ChromeDriver, etc.). Always use `driver.quit()` in `@AfterMethod/teardown` for complete cleanup. Forgetting `quit()` leaves zombie browser/driver processes consuming system resources. If multiple windows: either close each individually, or just call `quit()` to end the session.

◆ Advanced Selenium

Q10. How do you handle iframes?

■ Selenium cannot interact with elements inside an iframe without switching context. Switch by index: `driver.switchTo().frame(0)`. By name/id: `driver.switchTo().frame('frameName')`. By WebElement: `driver.switchTo().frame(driver.findElement(By.id('myFrame')))`. All `findElement` calls then target the iframe DOM. Return to main page: `driver.switchTo().defaultContent()`. For nested iframes: switch to each outer frame first. Always switch back to `defaultContent` after iframe interaction.

Q11. What is the Actions class?

■ Actions simulates complex user interactions. Methods: `moveToElement(element)` (hover), `click(element)`, `doubleClick(element)`, `contextClick(element)` (right-click), `clickAndHold(element)`, `dragAndDrop(source, target)`, `sendKeys(element, keys)`. Chain actions and call `perform()` at the end: `new Actions(driver).moveToElement(menu).click(menuItem).perform()`. Keyboard: `keyDown(Keys.CONTROL).sendKeys('a').keyUp(Keys.CONTROL)` for Ctrl+A. Requires non-headless browser for some actions.

Q12. What is JavaScriptExecutor?

■ `JavaScriptExecutor` executes JavaScript in the browser context: `JavaScriptExecutor js = (JavaScriptExecutor) driver`. Common uses: click hidden/overlapping elements (`js.executeScript('arguments[0].click()', element)`), scroll to element (`js.executeScript('arguments[0].scrollIntoView(true)', element)`), scroll page (`js.executeScript('window.scrollTo(0,500)')`), get values or set input values when `sendKeys` fails. Use as a fallback when standard WebDriver methods are unreliable.

Q13. How do you handle `StaleElementReferenceException`?

■ `StaleElementReferenceException` occurs when a `WebElement` reference becomes invalid because the DOM was rebuilt after the element was found. Solutions: 1) Re-find the element just before interacting (don't store element references long). 2) Try-catch with retry logic to re-find and retry. 3) `FluentWait` with `ignoringExceptions(StaleElementReferenceException.class)`. 4) `PageFactory @FindBy` re-finds on each use. Common cause: AJAX updates, page navigations, dynamic content refreshes.

Q14. What is the Page Object Model?

■ POM is a design pattern creating a separate class for each web page containing: locators (`By` or `@FindBy` fields), page methods (actions), and no test assertions. Tests use POM classes without knowing locator details. Benefits: separation of concerns, no code duplication, easy maintenance (locator changes in one place), improved readability. `LoginPage` (locators + `login()` method) used by `LoginTest` (test logic only). The gold standard for maintainable Selenium test architecture.

Q15. What is PageFactory?

■ `PageFactory` initializes `@FindBy` annotated `WebElement` fields lazily (searched on first use, not at construction). Use `PageFactory.initElements(driver, this)` in the constructor. `@FindBy(id='user')` private `WebElement` `userField`. Cleaner than repeated `driver.findElement()` calls. `@FindBy`s requires all conditions; `@FindAll` requires any. `PageFactory` re-searches the DOM on each access, partially mitigating `StaleElementReferenceException`. Use with POM pattern for clean test code.

Q16. What is Selenium Grid?

■ Selenium Grid enables parallel test execution across multiple machines and browsers simultaneously. Architecture: Hub (distributes tests) + Nodes (machines with browsers). Selenium 4 Grid 4 supports fully distributed mode, Docker integration, and improved scalability. Tests use `RemoteWebDriver` pointing to the Hub, which routes to appropriate Nodes. Use for: cross-browser testing, reducing execution time, and testing on different OS/browser combinations.

Q17. How do you take a screenshot?

■ Use `TakesScreenshot`: `TakesScreenshot ts = (TakesScreenshot) driver; File src = ts.getScreenshotAs(OutputType.FILE); FileUtils.copyFile(src, new File('screenshots/test.png'))`. Selenium 4+: `WebElement` `screenshot`: `element.getScreenshotAs(OutputType.FILE)`. In TestNG `@AfterMethod`: `if(result.getStatus()!=ITestResult.FAILURE) { takeScreenshot() }`. Allure/ExtentReports can embed screenshots directly in reports for failure analysis.

Q18. What is headless browser testing?

■ Headless browsers run without visible UI — ideal for CI/CD environments (no display required), faster execution, reduced resource usage. Chrome headless: `ChromeOptions opts = new ChromeOptions(); opts.addArguments('--headless=new'); driver = new ChromeDriver(opts)`. Firefox: `FirefoxOptions opts = new FirefoxOptions(); opts.addArguments('--headless')`. Limitations: some UI interactions may differ, JavaScript rendering issues possible. Chrome headless is the most widely used in CI pipelines.

Q19. How do you scroll a page in Selenium?

■ `JavascriptExecutor`: `js.executeScript('window.scrollTo(0,500)')` — scroll down 500px. `js.executeScript('window.scrollTo(0,document.body.scrollHeight)')` — scroll to bottom. `js.executeScript('arguments[0].scrollIntoView(true)',element)` — scroll element into view. Selenium 4 Actions: `new Actions(driver).scrollToElement(element).perform()` or `scrollByAmount(x,y)`. Keyboard: `body.sendKeys(Keys.END)`. `scrollIntoView` is most commonly used before clicking obscured elements.

Q20. What is the Robot class?

■ Java's Robot class (`java.awt.Robot`) simulates native OS-level keyboard and mouse events, useful for: handling OS-level file upload dialogs, Windows authentication dialogs, and keyboard shortcuts not handled by WebDriver. Cannot run headlessly (requires a display). For file input elements, prefer `driver.findElement(By.id("fileInput")).sendKeys(absoluteFilePath)` which works headlessly and avoids Robot class complexity entirely.

13. TestNG

◆ TestNG Essentials

Q1. What is TestNG and how is it different from JUnit?

■ TestNG is a testing framework inspired by JUnit with additional features: grouping (run subsets of tests), parallel execution configuration via XML, test dependencies (`dependsOnMethods`), `@DataProvider` for data-driven testing, `BeforeSuite/AfterSuite/BeforeTest/AfterTest` annotations spanning multiple classes, and flexible `testng.xml` configuration. JUnit 5 has narrowed the gap, but TestNG remains popular in enterprise Selenium frameworks.

Q2. What are TestNG annotations and their execution order?

■ Order: `@BeforeSuite` → `@BeforeTest` → `@BeforeGroups` → `@BeforeClass` → `@BeforeMethod` → `@Test` → `@AfterMethod` → `@AfterClass` → `@AfterGroups` → `@AfterTest` → `@AfterSuite`. `@BeforeSuite` runs once before all tests. `@BeforeTest` before all tests in a `<test>` XML tag. `@BeforeClass` before the first method in the class. `@BeforeMethod` before each `@Test`. `After*` annotations run in reverse order. `@DataProvider`, `@Factory`, `@Listeners` are non-lifecycle annotations.

Q3. What is the difference between `@BeforeMethod`, `@BeforeClass`, `@BeforeSuite`, `@BeforeTest`?

■ `@BeforeMethod`: runs before EACH `@Test` method — use for per-test setup (initialize `WebDriver`, navigate to URL). `@BeforeClass`: runs ONCE before the first test in the class — use for class-level setup (DB connection). `@BeforeTest`: runs before all methods in a `<test>` XML tag spanning multiple classes. `@BeforeSuite`: runs ONCE before all tests — use for global setup (start Grid, load config). `@After*` are the teardown counterparts in reverse order.

Q4. What are groups in TestNG?

■ Groups categorize tests for selective execution. Annotate: `@Test(groups={'smoke','regression'})`. Run in `testng.xml`: `<groups><run><include name='smoke'/></run></groups>`. Exclude: `<exclude name='slow'/>`. `@BeforeGroups` and `@AfterGroups` run setup/teardown for group execution. Use groups for: smoke tests on every build, full regression on nightly runs, performance-tagged tests separately. Groups enable CI/CD efficiency by running only relevant tests.

Q5. What is `@DataProvider`?

■ `@DataProvider` feeds parameterized data to `@Test` methods. Return `Object[][]` where each inner array is one test run's parameters. Example: `@DataProvider public Object[][] loginData() { return new Object[][]{{ 'admin','pass1'}, {'user','pass2'} }; }`. Reference in test: `@Test(dataProvider='loginData')`. `DataProvider` in external class: `@Test(dataProvider='data', dataProviderClass=DataClass.class)`. Enables data-driven testing without duplicating test methods. Supports parallel execution: `@DataProvider(parallel=true)`.

Q6. What is `testng.xml`?

■ `testng.xml` configures and orchestrates TestNG execution. Defines: suite name, test names, included/excluded classes and methods, groups to run/exclude, parallel settings (`parallel='methods'/'classes'/'tests'`, thread-count), parameters (`@Parameters` annotation in tests reads from here), listeners, and data providers. Multiple `testng.xml` files for different test purposes (smoke, regression). Maven Surefire plugin reads `testng.xml` to execute tests.

Q7. How do you set test priority?

■ `@Test(priority=1)` — lower number runs first. No-priority tests default to 0. Same-priority tests run in alphabetical/definition order. Priority is within a class; across classes the order may vary. Note: test interdependence via priority is a design smell — prefer `dependsOnMethods` for explicit dependencies. Write independent tests when possible. Priority is useful for ordering login before other tests within a single test class.

Q8. What is `dependsOnMethods`?

■ `@Test(dependsOnMethods={'loginTest'})` makes a test depend on another — if `loginTest` fails, the dependent test is SKIPPED (not failed). Can depend on multiple: `dependsOnMethods={'step1','step2'}`. `dependsOnGroups` for group-level dependencies. Creates tight coupling — design tests independently whenever possible. Best use: sequential scenarios where later steps genuinely require earlier ones (multi-step checkout flow). If the dependency passes, the dependent test runs normally.

Q9. What are soft assertions vs hard assertions?

■ Hard assertions (Assert class): fail immediately on first assertion failure — subsequent lines not executed. Use for critical single-point validations. Soft assertions (SoftAssert): record all failures but continue execution — call `sa.assertAll()` at end to report all. Usage: `SoftAssert sa = new SoftAssert(); sa.assertEquals(actual, expected); sa.assertAll()`. Soft assertions give a complete picture of all failures in one test run. Use for multi-field validation where you want to see all failures at once.

Q10. What is `ITestListener`?

■ `ITestListener` provides callbacks for TestNG events: `onTestStart`, `onTestSuccess`, `onTestFailure`, `onTestSkipped`, `onStart(ITestContext)`, `onFinish(ITestContext)`. Implement to: take screenshots on failure, log results to reporting tools, send notifications, update test management systems. Register in `testng.xml`: `<listeners><listener class-name='MyListener'/></listeners>` or `@Listeners(MyListener.class)` on test classes. Used by ExtentReports, Allure, and custom reporting integrations.

Q11. How do you configure parallel execution in TestNG?

■ In `testng.xml`: `<suite parallel='methods' thread-count='4'>`. Options: `methods` (each `@Test` in parallel), `classes` (all methods in a class run in a thread), `tests` (each `<test>` tag in parallel), `instances`. Critical: use `ThreadLocal<WebDriver>` to give each thread its own browser instance to prevent interference. `@DataProvider(parallel=true)` runs data iterations in parallel. Parallel execution significantly reduces total test suite run time.

Q12. What is `retryAnalyzer`?

■ `retryAnalyzer` auto-retries failed tests. Implement `IRetryAnalyzer`: `boolean retry(ITestResult result) { if(retryCount < maxRetry) { retryCount++; return true; } return false; }`. Apply: `@Test(retryAnalyzer=RetryAnalyzer.class)` or globally via `IAnnotationTransformer` listener. Useful for handling genuinely flaky tests (network timeouts, timing issues). Caution: retrying can mask real bugs — fix root causes first. Reports show both failed and retried attempts.

Q13. How do you handle expected exceptions?

■ `@Test(expectedExceptions=IllegalArgumentException.class)` passes if the specified exception is thrown — fails if no exception or a different one is thrown. Multiple: `expectedExceptions={NPE.class, IAE.class}`. With message validation: `expectedExceptionsMessageRegExp='.*invalid.*'`. Cleaner than try-catch in test code. Alternative: `Assert.assertThrows()` for more modern assertion style. The test only passes when the correct exception is thrown with the expected message.

Q14. What is invocationCount?

■ `@Test(invocationCount=5)` runs the same test method 5 times consecutively. Combine with `threadPoolSize` for parallel invocations: `@Test(invocationCount=10, threadPoolSize=3)` — 10 invocations across 3 threads simultaneously. Useful for: load-testing a specific operation, verifying thread safety, validating consistency of potentially flaky tests. `successPercentage`: `@Test(invocationCount=10, successPercentage=80)` passes if at least 80% of invocations succeed.

Q15. How do you generate reports in TestNG?

■ Default: HTML reports in `test-output/` directory (`index.html`, `emailable-report.html`). Extent Reports: implement `ITestListener` — `addTest`, `log`, `pass/fail/skip` with screenshots. Allure: add `allure-testng` dependency, run `allure serve target/allure-results` — provides interactive UI with timeline and behaviors. Cluecumber: Maven plugin generating standalone HTML from `cucumber.json`. Add listeners in `testng.xml`. Allure offers the richest interactive reporting; Extent Reports for commercial reporting needs.

14. RestAssured

◆ RestAssured Essentials

Q1. What is REST Assured?

■ REST Assured is a Java library for testing RESTful APIs with a fluent, BDD-style API built on Apache HttpClient. It simplifies HTTP request creation and response validation, integrating seamlessly with JUnit and TestNG. Supports all HTTP methods, authentication (Basic, OAuth2, Bearer), JSON/XML path extraction, schema validation, and logging. The most popular Java library for API test automation.

Q2. What is the BDD syntax in REST Assured?

■ REST Assured uses Given-When-Then: `given()`: sets preconditions (headers, auth, params, body, content type). `when()`: specifies HTTP method and endpoint (`get('/api/users')`, `post('/users')`). `then()`: validates response (`statusCode(200)`, `body('name', equalTo('John'))`). Example: `given().header('Accept','application/json').when().get('/api/users/1').then().statusCode(200).body('name', equalTo('Alice'))`.

Q3. How do you send GET and POST requests?

■ GET: `given().baseUrl('http://api.example.com').when().get('/users').then().statusCode(200)`. POST with JSON string: `given().contentType(ContentType.JSON).body("{\"name\":\"John\""}).when().post('/users').then().statusCode(201)`. POST with POJO (auto-serialized by Jackson): `User user = new User('John',30); given().contentType(ContentType.JSON).body(user).when().post('/users')`. Set globally: `RestAssured.baseURI = 'http://api.example.com'`.

Q4. How do you validate status code and response body?

■ Status: `.then().statusCode(200)` or `statusCode(equalTo(200))`. Body with Hamcrest matchers: `.body('field', equalTo('value'))`, `.body('list.size()', is(3))`, `.body('users.name', hasItem('Alice'))`, `.body('count', greaterThan(0))`. Chain multiple `.body()` calls for multiple assertions. Header check: `.header('Content-Type', containsString('json'))`. Extract value: `.extract().path('id')`. Full response: `Response r = .extract().response()`.

Q5. What is JSONPath in REST Assured?

■ JSONPath is a query language for JSON extraction. In assertions: `.body('user.name', equalTo('John'))` — nested field. `.body('users[0].id', is(1))` — first array element. `.body('users.size()', is(3))` — array size. `.body('users.findAll{u->u.age>25}.name', hasItems('Alice'))` — filter (Groovy GPath). Standalone: `JsonPath jp = response.jsonPath(); jp.getString('user.name');`; `jp.getList('users');`; `jp.getInt('users[0].id')`.

Q6. How do you handle authentication?

■ Basic: `given().auth().basic('user','pass')`. Preemptive basic (sends without challenge): `given().auth().preemptive().basic('user','pass')`. Bearer token: `given().header('Authorization','Bearer '+token)`. OAuth 2: `given().auth().oauth2(accessToken)`. Digest: `given().auth().digest('user','pass')`. Session/cookie: `given().cookie('JSESSIONID', value)`. OAuth 1: `given().auth().oauth(consumerKey, consumerSecret, token, tokenSecret)`.

Q7. What is RequestSpecification and ResponseSpecification?

■ RequestSpecification (RequestSpecBuilder) defines reusable request configuration: base URI, headers, auth, content type. Build once, reuse: RequestSpecification spec = new RequestSpecBuilder().setBaseUri(url).addHeader('Authorization','Bearer token').build(). Use: given().spec(spec).when().get('/users'). ResponseSpecification (ResponseSpecBuilder) defines common response assertions: new ResponseSpecBuilder().expectStatusCode(200).expectContentType(JSON).build(). Use in .then().spec(resSpec). Reduces duplication.

Q8. How do you extract values from a response?

■ Extract by path: String name = response.then().extract().path('user.name'). Full response: Response res = given().when().get('/user/1').then().extract().response(); then: res.jsonPath().getString('name'), res.statusCode(), res.getHeader('X-Total'). Extract as POJO: User user = response.then().extract().as(User.class). Extract list: List<String> names = response.then().extract().path('users.name'). Chain extraction: String id = given().body(data).when().post('/users').then().extract().path('id').

Q9. How do you perform JSON schema validation?

■ Add json-schema-validator dependency. Create schema file (schema.json) in src/test/resources. Validate: given().when().get('/user/1').then().body(matchesJsonSchemaInClasspath('schema.json')). Example schema: {type:'object', required:['id','name'], properties:{id:{type:'integer'},name:{type:'string'}}}. Schema validation catches structural contract violations automatically — API response must match defined types and required fields. Use JsonSchemaFactory for custom configurations.

Q10. How do you log requests and responses?

■ Log request: given().log().all() (everything), .log().headers(), .log().body(), .log().params(). Log response: .then().log().all(), .log().ifError() (only on failure), .log().ifValidationFails() (preferred — logs only when assertion fails). Global: RestAssured.filters(new RequestLoggingFilter(), new ResponseLoggingFilter()). Log to file: PrintStream out = new PrintStream('log.txt'); given().log().all(out). Essential for debugging failures in CI.

Q11. How do you serialize and deserialize POJOs?

■ Serialization (Object to JSON in request body): given().contentType(ContentType.JSON).body(userObject) — Jackson serializes automatically. Deserialization (JSON to Object from response): User user = when().get('/users/1').then().extract().as(User.class). For lists: List<User> users = when().get('/users').then().extract().jsonPath().getList('.', User.class). Add jackson-databind (or gson) to classpath. Use @JsonIgnoreProperties(ignoreUnknown=true) for partial mapping.

Q12. What is relaxedHTTPSValidation?

■ RestAssured.useRelaxedHTTPSValidation() or given().relaxedHTTPSValidation() disables SSL/TLS certificate validation — accepts self-signed or expired certificates. ONLY for test/dev environments with self-signed certs. NEVER use against production APIs — it bypasses security checks and makes the client vulnerable to man-in-the-middle attacks. For proper SSL in test environments: add the cert to a trust store and configure RestAssured to use it via SSLConfig.

Q13. How do you integrate REST Assured with TestNG?

■ Add rest-assured and testng dependencies. Test class: public class UserApiTest { @BeforeClass void setup() { RestAssured.baseURI = 'http://api.example.com'; RestAssured.port = 8080; } @Test void testGetUser() { given().when().get('/users/1').then().statusCode(200).body('name', equalTo('Alice')); } }. Run via testng.xml or Maven. Use @DataProvider for parameterized API tests. Use SoftAssert for validating multiple response fields in one test.

Q14. How do you set query and path parameters?

■ Query params: `given().queryParams('status','active').queryParams('page',1).when().get('/users')`. Multiple at once: `given().params('status','active','page',1)`. Path params: `given().pathParam('userId',42).when().get('/users/{userId}')`. Multiple path params: `given().pathParams('org','myorg','repo','myrepo').when().get('/orgs/{org}/repos/{repo}')`. Query params appear after `?` in URL; path params substitute placeholders in URL template.

Q15. What is the base URI and base path?

■ Base URI is the root URL: `RestAssured.baseURI = 'https://api.example.com'` (global) or `given().baseUri(url)`. Base path is a common prefix: `RestAssured.basePath = '/v1/api'`. Port: `RestAssured.port = 8080`. Combined full URL: `baseURI + basePath + endpoint path`. Set globally in `@BeforeClass` to avoid repetition across tests. Reset to defaults with `RestAssured.reset()` in `@AfterClass`. Saves having to repeat the server URL in every test method.

15. SoapUI

◆ SoapUI Essentials

Q1. What is SoapUI?

■ SoapUI (SmartBear ReadyAPI) is a testing tool for SOAP and REST web services. Capabilities: functional testing (test suites/cases), load/performance testing, security scanning, service mocking, data-driven testing, and CI/CD integration. SoapUI Open Source is free. ReadyAPI adds performance, security, and virtualization. Widely used in enterprise environments where SOAP/WSDL-based services are prevalent.

Q2. What is the difference between SOAP and REST?

■ SOAP: uses XML exclusively, defined by WSDL contract, protocol-agnostic (HTTP/SMTP/TCP), built-in error handling (Fault elements), WS-Security, WS-Reliability for guaranteed delivery — heavyweight enterprise protocol. REST: uses HTTP natively, lightweight (JSON/XML/text), stateless, resource-based URLs, standard HTTP methods (GET/POST/PUT/DELETE), optional OpenAPI contract — simpler and more widely used for modern APIs. REST is dominant for new development; SOAP persists in banking/government/enterprise.

Q3. What is WSDL and how do you import it?

■ WSDL (Web Services Description Language) is an XML document describing a SOAP service: operations, message formats, data types, bindings, and endpoints. Import in SoapUI: File → New SOAP Project → enter WSDL URL or file path → SoapUI auto-generates request templates for all operations. SoapUI parses the WSDL and creates service, binding, port, and operation nodes with pre-populated XML request templates. WSDL is the service contract.

Q4. How do you create test suites and test cases?

■ Right-click project → Add TestSuite. Within TestSuite → Add TestCase. Within TestCase → Add Step → choose type (Request, Groovy Script, Property Transfer, JDBC, etc.). Add assertions to each request step (right-click response → Add Assertion). Organize: multiple TestSuites (Smoke, Regression), multiple TestCases per suite (CRUD operations), multiple steps per case. Run from TestSuite view or integrate via CLI/Maven.

Q5. What are assertion types in SoapUI?

■ Property Content: Contains, Not Contains, XPath Match, XQuery Match, JsonPath Match, JsonPath Count, JsonPath Existence. Compliance/Standards: Schema Compliance (validates against WSDL schema), WS-Security Status. SLA: Response SLA (maximum response time). Script: Groovy script for custom logic assertions. HTTP: Valid HTTP Status Codes, HTTP Response. JDBC: verify database state after API call. Combine multiple assertions for comprehensive request validation.

Q6. What is a property transfer step?

■ Property Transfer extracts values from one response and passes them to another request — essential for chaining API calls. Example: extract auth token from login response → inject into Authorization header of next request. Configure: Source (step name, property, XPath/JsonPath to extract), Target (step name, property where value goes). Multiple transfers in one step. Also used for storing created resource IDs (POST → GET/DELETE chaining).

Q7. How do you use XPath in SoapUI?

■ In XPath Match assertions: Add Assertion → XPath Match → enter expression (`//ns:userId/text()`) and expected value. In Property Transfer: Source Path type = XPath, enter expression. In Groovy scripts: `def holder = new XmlHolder(messageExchange.responseContentAsXml); holder.getNodeValue('//ns:result')`. Declare XML namespaces in SoapUI's project namespace mappings for proper prefix resolution. XPath 1.0 supported. JsonPath used for REST responses.

Q8. How do you parameterize requests?

■ Use property expansion: `${#TestCase#propertyName}` references TestCase properties. `${#Project#env}` for project properties. Set via Test Properties panel or Groovy script (`testRunner.testCase.setPropertyValue('id','123')`). DataSource step reads from Excel/CSV/database — columns map to properties. DataSource Loop iterates through all data rows. This enables data-driven testing: same steps execute with multiple data sets. CLI: pass properties with -P flag.

Q9. What is a MockService?

■ MockService simulates a web service without needing the real backend. Create: right-click operation → Add Mock Response → start MockService. Configure: response XML/JSON, status code, response delay, and dynamic responses via Groovy scripts. REST MockService can simulate entire APIs. Use cases: parallel development (frontend doesn't wait for backend), testing edge cases (errors, timeouts) that are hard to reproduce with real services, and component isolation.

Q10. How do you run SoapUI tests from command line?

■ Use `testrunner.sh` (Linux/Mac) or `testrunner.bat` (Windows) from SoapUI bin directory. Command: `testrunner.sh -s 'TestSuite1' -c 'TestCase1' -r -f /reports /path/to/project.xml`. Options: -s (suite name), -c (test case), -e (endpoint override), -p (port), -P (property: `-Penv=staging`), -r (print report), -f (report folder). Integrate with Jenkins as shell step. Maven: `soapui-maven-plugin`. JUnit-format reports can be displayed in Jenkins.

Q11. What is Groovy scripting in SoapUI?

■ SoapUI uses Apache Groovy for: Script test steps (custom logic, loops), Script assertions (custom validation), event handlers, and DataSource processing. Access to: `messageExchange` (request/response content), `testRunner` (control execution), `testCase`, `testSuite`, `project`, `log`, and `context` (shared data). Example: `def json = new groovy.json.JsonSlurper().parseText(messageExchange.responseContent); assert json.status == 'success'`. Enables complex logic beyond GUI capabilities.

Q12. How do you handle authentication in SoapUI?

■ Basic auth: request editor → Auth tab → Basic → enter credentials. SOAP WS-Security: project-level Outgoing WS-Security profiles → UsernameToken. OAuth 2.0: Auth tab → OAuth 2.0 → configure grant type, token endpoint, client ID/secret → Get Access Token. Bearer token: add Authorization header manually. For all project requests: set project-level outgoing security profiles. Use project properties for credentials to switch between environments easily.

Q13. What is a load test in SoapUI?

■ Load tests simulate concurrent users to evaluate performance. Create: right-click TestCase → New LoadTest. Configure: Strategy (Simple — fixed threads, Burst — variable), thread count, test limit (duration or request count), think time between requests. Assertions: Response SLA (max acceptable response time — test fails if exceeded). Results: requests/second, min/max/avg response times, error count. SoapUI OS for basic load testing; ReadyAPI Performance for advanced scenarios.

Q14. How do you integrate SoapUI with CI/CD?

■ Jenkins: SoapUI plugin or execute testrunner.sh as shell step, parse JUnit XML reports for results display. Maven: soapui-maven-plugin in pom.xml, run with mvn soapui:test, pass environment properties (-Pbase.url=https://staging.api.com). Docker: containerize SoapUI for CI. Pass environment-specific endpoints, credentials, and properties as CLI args or Maven properties. Store test reports as Jenkins artifacts. ReadyAPI has native CI/CD agents for TeamCity, Bamboo, and Azure DevOps.

16. Cucumber

◆ BDD with Cucumber

Q1. What is Cucumber and BDD?

■ BDD (Behavior-Driven Development) emphasizes collaboration between developers, QA, and business stakeholders by describing behavior in plain language. Cucumber is a BDD tool that allows writing test scenarios in Gherkin (plain English DSL) which are then automated via step definitions. It bridges business requirements and automated tests, creating living documentation that is both specification and test.

Q2. What is Gherkin?

■ Gherkin is Cucumber's structured plain-text language for writing scenarios. Keywords: Feature, Background, Scenario, Given (preconditions), When (actions), Then (expected outcomes), And/But (additional steps), Scenario Outline (parameterized template), Examples (data for Scenario Outline), Doc Strings (multi-line text), Data Tables. Comments start with #. Business analysts and stakeholders can read and write Gherkin without programming knowledge.

Q3. What are Feature files?

■ Feature files (.feature extension) contain Gherkin test scenarios describing application behavior from the user's perspective. Structure: Feature keyword + description, then one or more Scenarios. Non-technical — stakeholders can read and contribute. They serve as living documentation — both specification and automated test. Stored in src/test/resources. Each feature file typically covers one user story or feature area. Cucumber maps each step to a Java step definition method.

Q4. What is the difference between Scenario and Scenario Outline?

■ Scenario describes a single test case with hardcoded values. Scenario Outline is a parameterized template with <variable> placeholders, driven by an Examples table — Cucumber runs it once per data row. Example: Scenario Outline: Login with <username> | Given user enters '<username>' and '<password>' | Examples: | username | password | | admin | pass1 |. Scenario Outline eliminates data-driven duplication. Multiple Examples tables with different tags can provide different datasets.

Q5. What are step definitions?

■ Step definitions are Java methods annotated with @Given, @When, @Then, @And, @But that match Gherkin steps via regex or Cucumber expressions. Cucumber maps each feature file step to a matching method at runtime. Parameters captured via regex groups or typed Cucumber expressions. State shared between steps via instance variables or DI (PicoContainer). Step definitions should be thin — delegate to page objects or service classes, not contain test logic.

Q6. What is a Runner class?

■ The Runner class triggers Cucumber execution. JUnit 4: @RunWith(Cucumber.class) @CucumberOptions(features='src/test/resources/features', glue='step_definitions', plugin={'pretty','html:target/reports'}). TestNG: extend AbstractTestNGCucumberTests. The runner connects Cucumber to the test framework, enabling Maven/Gradle execution and CI/CD integration. Multiple runners can target different feature sets or tags for selective execution.

Q7. What are Cucumber tags?

■ Tags categorize scenarios for selective execution. Syntax: @smoke before Feature or Scenario. Run with: @CucumberOptions(tags='@smoke') or --tags '@smoke and not @slow'. Tag expressions: @smoke and @regression, @smoke or @ui, not @slow. Common tags: test type (@smoke, @regression), feature area (@login, @payment), status (@wip, @known-bug), environment (@mobile, @desktop). In Jenkins/Maven: pass tags as properties for CI/CD selective runs.

Q8. What is a Background?

■ Background defines steps that run before EACH scenario in a Feature file — like a @BeforeMethod but in Gherkin. Example: Background: Given user is logged in And user is on the dashboard. All scenarios in that feature share this setup without repetition. Runs after @Before hooks. Use Background for state common to ALL scenarios in a file. For scenarios needing different preconditions, separate them into different feature files.

Q9. What are Cucumber hooks?

■ @Before runs before each scenario (setup: init WebDriver, test data). @After runs after each scenario (teardown: screenshot on failure, quit driver). @BeforeStep and @AfterStep around each step (detailed logging). @Before(order=1) controls order (lower = runs first for @Before, higher = runs first for @After). Hooks can be tagged: @Before('@login') only runs for @login scenarios. Multiple hook methods allowed. Always quit WebDriver in @After.

Q10. How do you handle data tables?

■ Data Tables pass structured data to steps. Inline: | Name | Age | | Alice | 25 |. Access as DataTable: .asLists() (List<List<String>>), .asMaps() (List<Map<String,String>>), .asMap() (single Map). Transform to POJO: .asList(User.class) using DataTableType transformers. Example step: When user creates users with table → receives DataTable parameter. Use for structured input data; Doc Strings for multi-line text like JSON/XML payloads.

Q11. What is @CucumberOptions?

■ @CucumberOptions configures Cucumber: features (path to .feature files), glue (package with step definitions/hooks), plugin (output formats: pretty, html:path, json:path, junit:path, allure-results), tags (filter by tags), dryRun (check step definition coverage without executing — finds missing step definitions), monochrome (cleaner console output). Example: @CucumberOptions(features='src/test/resources', glue='com.example.steps', plugin={'pretty','html:target/reports'}, tags='@smoke').

Q12. How do you pass data between steps?

■ Via instance variables in the same step definition class (one instance per scenario). Via dependency injection with PicoContainer (add cucumber-picocontainer): inject a shared context class into multiple step definition classes — the same instance is shared within a scenario. Via Scenario object: scenario.log('message'). Avoid static variables — cause parallel execution issues. PicoContainer is the recommended approach for cross-class state sharing.

Q13. How do you integrate Cucumber with Selenium?

■ 1) Add cucumber-java, cucumber-testng, selenium-java. 2) Initialize WebDriver in @Before hook. 3) Store driver in shared context class (injected via PicoContainer constructor injection). 4) Step definitions receive context via constructor and use driver for browser interactions. 5) Page Objects accept driver from context. 6) @After hook: take screenshot on failure (via scenario.isFailed()), call driver.quit(). This structure keeps WebDriver lifecycle and test steps cleanly separated.

Q14. What is dry run?

■ `@CucumberOptions(dryRun=true)` scans feature files and step definitions to check coverage without executing anything. Prints pending step snippets for unimplemented steps. Useful: validate coverage before running full suite, generate step definition templates for new features, check feature file syntax. No browser or API calls made. Very fast. Run after writing new feature files to identify missing implementations before the full test run.

Q15. How do you generate reports in Cucumber?

■ Built-in: `plugin={'html:target/cucumber-reports', 'json:target/cucumber.json', 'junit:target/junit.xml'}`. Extent Reports: implement `EventListener`, configure `ExtentCucumberAdapter` with `extent.properties`. Allure: add `allure-cucumber6-jvm` dependency, add `AllureCucumber6Jvm` to plugins, generate with `allure serve target/allure-results`. Cluecumber Maven plugin from `cucumber.json`. Reports include: pass/fail status, step details, screenshots (attach in `@After`), tags, and duration.

17. Jenkins

◆ Jenkins CI/CD

Q1. What is Jenkins?

■ Jenkins is an open-source automation server for building, testing, and deploying software. It implements CI (automatically build/test on every commit) and CD (automated release pipeline). Jenkins orchestrates: compiling code, running tests, code quality checks, building Docker images, deploying to environments, and sending notifications. Supports 1800+ plugins for integration with virtually every DevOps tool.

Q2. What is CI/CD and how does Jenkins help?

■ CI (Continuous Integration): frequent code merges trigger automated build and tests — catches integration issues early. CD (Continuous Delivery): automatically deploys tested code to staging. CD (Continuous Deployment): every passing build auto-deploys to production. Jenkins implements this by: triggering builds on git commits (webhooks/polling), running test suites, failing builds on test failures, and orchestrating deployment scripts — creating a reliable, repeatable delivery pipeline.

Q3. What is the difference between Freestyle and Pipeline projects?

■ Freestyle project: GUI-based configuration, limited to sequential steps, pipeline config not version-controlled — simple for basic builds. Pipeline project: pipeline defined as code (Jenkinsfile in SCM), supports complex workflows with stages/parallel/conditionals/loops, version-controlled, reproducible, auditable. Jenkinsfile can be Declarative (structured, easier) or Scripted (Groovy, more flexible). Pipelines are the modern standard; Freestyle is legacy.

Q4. What is a Declarative vs Scripted pipeline?

■ Declarative: structured syntax with predefined sections — `pipeline { agent any; stages { stage('Build') { steps { sh 'mvn compile' } } } post { failure { mail to:'team@co.com' } } }`. Validates syntax before execution, has Snippet Generator. Scripted: full Groovy DSL in `node{}` blocks — any Groovy construct, maximum flexibility. Declarative is recommended for most pipelines; use Scripted only when Declarative cannot express the needed logic.

Q5. What is a Jenkinsfile?

■ A Jenkinsfile is a text file in the repository root defining the Jenkins pipeline. Benefits: pipeline as code (version-controlled, peer-reviewed, auditable), reusable across branches, enables Multibranch Pipeline auto-discovery. Each branch can have its own Jenkinsfile. Jenkins reads the file on webhook/poll trigger. Supports both Declarative and Scripted syntax. The presence of a Jenkinsfile is sufficient to auto-configure the build job.

Q6. What are stages and steps?

■ Stages represent logical phases of the pipeline: Build, Test, Security Scan, Docker Build, Deploy Staging, Integration Tests, Deploy Production. Each stage contains steps — actual commands: `sh 'mvn package'`, `bat 'mvn.cmd test'` (Windows), `echo 'message'`, `git 'https://...'`, `archiveArtifacts`, `publishHTML`. Stages appear as columns in the Stage View UI, making it easy to identify which phase failed and how long each took.

Q7. What are Jenkins agents and nodes?

■ The Jenkins controller manages configuration, scheduling, and UI — should not run builds. Agents execute builds. Types: Permanent agents (always-on machines connected via SSH or JNLP), Cloud agents (dynamically provisioned: EC2, Kubernetes pods, Docker containers), controller agent (not recommended for production). In Declarative: agent any (any available), agent { label 'linux' } (by label), agent { docker { image 'maven:3.8' } } (Docker container). Labels group agents by capability.

Q8. What are build triggers?

■ SCM Polling: Jenkins periodically checks for new commits (H/5 * * * * = every 5 min). Webhooks (Push-based): GitHub sends HTTP POST to Jenkins on commit — immediate, efficient. Build Periodically: cron schedule regardless of changes (nightly builds). Trigger from another job: post-build 'Build other projects' or pipeline trigger step. Manual: Build Now button. GitHub Branch Source plugin triggers on branch/PR creation automatically.

Q9. What are Jenkins environment variables?

■ Built-in: BUILD_NUMBER, BUILD_URL, JOB_NAME, WORKSPACE, BRANCH_NAME, GIT_COMMIT, GIT_BRANCH. Access: env.BUILD_NUMBER or just BUILD_NUMBER. Define custom: environment { MY_VAR = 'value' } block in Declarative, or withEnv(['VAR=value']) { }. From credentials: withCredentials([string(credentialsId:'api-key',variable:'KEY')]) { sh 'curl -H Authorization:\${KEY}' }. Pipeline parameters also available as params.MY_PARAM.

Q10. How do you use credentials securely?

■ Store credentials in Jenkins: Manage Jenkins → Credentials → add Secret Text, Username/Password, SSH Key, etc. Access in pipeline: withCredentials([usernamePassword(credentialsId:'db-creds',usernameVariable:'DB_USER',passwordVariable:'DB_PASS')]) { sh 'connect \$DB_USER \$DB_PASS' }. Values are automatically masked in logs. Never hardcode credentials in Jenkinsfile or pipeline code. Organization credentials can be stored globally and restricted by domain.

Q11. What is the post block?

■ The post block defines actions after all stages complete. Conditions: always (always runs), success (only on success), failure (only on failure), unstable, changed (result differs from previous build), fixed (was failing, now passes), regression (was passing, now fails), aborted, cleanup (always runs last). Example: post { always { junit 'target/surefire-reports/*.xml' } failure { email body:'Build failed', to:'team@co.com' } success { archiveArtifacts 'target/*.jar' } }

Q12. What are Jenkins shared libraries?

■ Shared libraries provide reusable Groovy code across Jenkinsfiles. Structure in a separate Git repo: vars/ (global callable scripts), src/ (Groovy classes), resources/ (non-Groovy files). Register in Jenkins: Manage Jenkins → Configure System → Global Pipeline Libraries. Import in Jenkinsfile: @Library('my-lib')_. Define vars/deployApp.groovy with def call(Map config) {} — use as deployApp(env:'staging') in any pipeline. Standardizes CI/CD practices organization-wide.

Q13. How do you run parallel stages?

■ Declarative: stages { stage('Parallel') { parallel { stage('Unit Tests') { steps { sh 'mvn test' } }, stage('Integration Tests') { steps { sh 'mvn verify -P integration' } } } }. failFast: true inside parallel aborts all branches if any fails. Scripted: parallel(unitTests: { node { sh 'mvn test' } }, integrationTests: { node { sh 'mvn verify' } }). Parallel stages run simultaneously — ideal for cross-browser Selenium, multi-platform builds, or independent test suites.

Q14. How do you integrate Jenkins with GitHub?

■ Webhook approach (preferred): GitHub → Settings → Webhooks → add Jenkins URL (<http://jenkins/github-webhook/>) → Jenkins GitHub plugin processes it and triggers builds. SCM polling: configure repository URL and poll schedule. Multibranch Pipeline with GitHub Branch Source plugin: auto-discovers branches and PRs, creates jobs automatically. GitHub Status API: Jenkins reports build status back to GitHub (commit checks). Requires network access between GitHub and Jenkins.

Q15. What are build parameters?

■ Parameters allow user input before a build or parameterization from upstream. Types: string, boolean (checkbox), choice (dropdown), password (masked), file (upload). Define in Declarative: `parameters { string(name:'BRANCH',defaultValue:'main') choice(name:'ENV',choices:['dev','staging','prod']) }`. Access: `params.BRANCH`, `params.ENV`. Trigger parameterized builds: `build job:'deploy', parameters:[string(name:'ENV',value:'prod')]`. Enable 'This project is parameterized' in Freestyle projects.

Q16. What is withCredentials?

■ `withCredentials` binds credentials to variables within a block scope — values are never written to disk or logged (Jenkins auto-masks). Bindings: `string(credentialsId,variable)`, `usernamePassword(credentialsId,usernameVariable,passwordVariable)`, `sshUserPrivateKey(credentialsId,keyFileVariable,usernameVariable)`, `file(credentialsId,variable)`. Best practice: keep blocks as narrow as possible to minimize credential exposure scope. Credentials are disposed of when the block exits.

Q17. How do you configure Jenkins with Maven?

■ 1) Install Maven: Manage Jenkins → Global Tool Configuration → Maven → Add Maven (name, version auto-installed). 2) Freestyle: Build step → Invoke Maven Targets, select Maven, set goals (clean install). 3) Pipeline: `tools { maven 'Maven_3.8' }` then `sh 'mvn clean install'`. 4) Maven settings.xml: Config File Provider plugin → Managed Files → add settings.xml with repository credentials → reference with `withMaven(maven:'Maven_3.8', mavenSettingsConfig:'settings-id')`. 5) Maven wrapper (mvnw) works without global installation.

18. Docker

◆ Docker Basics

Q1. What is Docker?

■ Docker is a containerization platform packaging applications and dependencies into lightweight, portable containers that run consistently across any environment — solving the 'works on my machine' problem. Containers share the host OS kernel, start in milliseconds, use minimal memory (MBs vs GBs for VMs), and are isolated via Linux namespaces and cgroups. Enables: environment consistency, microservices, dependency isolation, and efficient CI/CD pipelines.

Q2. What is the difference between a container and a VM?

■ VMs virtualize hardware — each has a full OS, kernel, and dependencies managed by a hypervisor. Heavy (GBs), slow to start (minutes), strong isolation (separate kernel). Containers virtualize at the OS level — share the host kernel, contain only app + dependencies. Lightweight (MBs), fast (milliseconds), process-level isolation via namespaces/cgroups. VMs for full OS isolation requirements; containers for microservices, CI/CD, and efficient resource utilization.

Q3. What is a Docker image?

■ A Docker image is a read-only template for creating containers, containing: application code, runtime, libraries, environment variables, and configuration. Built in layers (each Dockerfile instruction adds a cached layer). Layers are shared between images, saving disk space and build time. Images stored in registries (Docker Hub, ECR, GCR). Naming: repository:tag (nginx:1.25, myapp:v1.0). Images are immutable; containers are running instances of images.

Q4. What is a Dockerfile and common instructions?

■ A Dockerfile is a text file with build instructions. Common: FROM (base image — always first), RUN (execute command during build — creates a layer), COPY (copy files from build context), ADD (like COPY + tar extraction + URL support), WORKDIR (set working directory), ENV (environment variable), EXPOSE (document port — does not publish), CMD (default command — overridable), ENTRYPOINT (fixed executable), USER (set user), ARG (build-time variable), VOLUME (mount point), LABEL (metadata).

Q5. What is the difference between CMD and ENTRYPOINT?

■ ENTRYPOINT defines the main executable that always runs — only overridable with --entrypoint flag. CMD provides default arguments to ENTRYPOINT or default command if no ENTRYPOINT — easily overridden by docker run args. Best pattern: ENTRYPOINT ['java','-jar'] CMD ['app.jar'] — java -jar always runs, app.jar can be overridden: docker run myimage other.jar. Use exec form ([]) over shell form (string) — direct process, no shell overhead, proper signal handling.

Q6. What is the difference between COPY and ADD?

■ COPY is simpler and preferred — copies files/directories from build context to image. ADD has all COPY features plus: auto-extracts tar archives and supports remote URLs as source. Best practice: use COPY by default for predictability. Use ADD only when you need tar extraction. Remote URL downloads in ADD are not cached and have security concerns — prefer RUN curl instead. Docker linting tools (hadolint) flag ADD where COPY would suffice.

Q7. What are Docker volumes?

■ Volumes persist data beyond container lifecycle — data survives container removal. Types: Named volumes (docker volume create mydata, managed by Docker, stored in /var/lib/docker/volumes/) and anonymous volumes (no name, harder to manage). Mount: docker run -v mydata:/app/data. In Compose: volumes: mydata:/app/data. Volumes are faster than bind mounts on Mac/Windows (no file sync overhead). Use for: database files, app state, logs, shared data between containers.

Q8. What is the difference between volumes and bind mounts?

■ Volumes: managed by Docker, stored in Docker's directory, platform-independent, no host directory dependency — best for production persistent data. Bind mounts: mount a specific host path into the container (docker run -v /host/path:/container/path) — reflects host filesystem changes directly in container. Best for development (mount source code for live reload/hot reload). Bind mounts are less portable; named volumes recommended for production.

Q9. What is the difference between docker run and docker start?

■ docker run creates a NEW container from an image and starts it — accepts all configuration (ports -p, volumes -v, env vars -e, network). docker start starts an EXISTING stopped container — uses the original docker run configuration, no new options accepted. docker stop sends SIGTERM then SIGKILL. docker restart = stop + start. Use docker run to create new containers; docker start/stop to manage existing ones.

◆ Networking, Compose & Advanced

Q10. What are Docker network types?

■ Bridge (default): containers on same bridge network communicate by container name; isolated from host. Custom bridge networks provide DNS resolution — always create custom bridges rather than using default. Host: container shares host network stack — no isolation, better performance, Linux only. Overlay: connects containers across multiple Docker hosts (Swarm/Kubernetes). None: no networking, fully isolated. Macvlan: assigns MAC address for direct physical network access.

Q11. What is Docker Compose?

■ Docker Compose defines and runs multi-container applications with a docker-compose.yml file specifying services (containers), networks, and volumes. Services define: image or build context, ports, environment variables, volumes, depends_on, restart policy. Run: docker compose up -d. Stop and remove: docker compose down. Example: web service (Spring Boot) + db (MySQL) + cache (Redis) — Compose creates them in the right order and networks them together automatically.

Q12. What is the difference between docker compose up and docker compose start?

■ docker compose up creates and starts containers — if containers don't exist, creates them from scratch; also builds images if build is defined (or use --build flag). docker compose start starts existing stopped containers without creating new ones — no config changes applied. docker compose up is for initial startup and after configuration changes; docker compose start to resume previously stopped services.

Q13. What is a multi-stage build?

■ Multi-stage builds use multiple FROM instructions to create intermediate build stages, copying only needed artifacts to the final image — dramatically reduces image size. Example: Stage 1 (builder): FROM maven:3.8 AS builder; COPY . .; RUN mvn package. Stage 2 (runtime): FROM eclipse-temurin:17-jre-slim; COPY --from=builder /app/target/app.jar app.jar; CMD java -jar app.jar. Final image has no Maven, source code, or build tools — only JRE and JAR. Essential for production images.

Q14. What is Docker Swarm?

■ Docker Swarm is Docker's native orchestration for managing a cluster of Docker nodes. Swarm consists of manager nodes (orchestration, scheduling) and worker nodes (running containers). Services define desired state: `docker service create --replicas 3 myapp`. Swarm distributes replicas across nodes, handles failover, and enables rolling updates. Simpler than Kubernetes but less feature-rich. Activated with `docker swarm init` on the manager node.

Q15. What is the difference between EXPOSE and port mapping?

■ EXPOSE in Dockerfile documents that the container listens on a port — it does NOT publish the port or make it externally accessible. It is metadata/documentation only. Port mapping (`-p host:container` in `docker run`) actually binds the host port to the container port: `docker run -p 8080:80 nginx` — accessible at `localhost:8080`. Without `-p`, the port is only accessible within the Docker network. EXPOSE enables `docker run -P` (publish all exposed ports to random host ports).

Q16. How do you view container logs?

■ `docker logs <container>` shows STDOUT/STDERR. Options: `-f` or `--follow` (stream live like `tail -f`), `--tail 100` (last 100 lines), `--since 1h` (last hour), `--timestamps`. Docker Compose: `docker compose logs -f web`. Applications should log to STDOUT/STDERR (not files) to integrate with Docker's logging system. Configure log drivers (json-file, awslogs, splunk) in `daemon.json` or per container. Use centralized logging (ELK, CloudWatch) for production aggregation.

Q17. What is docker exec?

■ `docker exec` runs a command in a RUNNING container. Open interactive shell: `docker exec -it mycontainer /bin/bash` (or `/bin/sh` for Alpine). Run specific command: `docker exec mycontainer ls /app`. Run as root: `docker exec -u root mycontainer bash`. Useful for debugging running containers without SSH. Changes made via `exec` are NOT persisted when the container is removed (unless written to a volume). For production debugging, prefer `docker logs` first.

Q18. What is .dockerignore?

■ `.dockerignore` specifies files to exclude from the Docker build context sent to the daemon. Similar to `.gitignore`. Common exclusions: `.git`, `target/` or `node_modules/` (build outputs), `*.log`, `.env` (secrets), `tests/`, `docs/`. Reduces build context size (speeds up builds), prevents sensitive files from entering images, and improves build cache efficiency. Always create `.dockerignore` — without it, the entire directory is sent as build context, which can be extremely slow for large projects.

Q19. How do you clean up Docker resources?

■ `docker system prune`: removes stopped containers, dangling images, unused networks, and build cache. Add `-a` for all unused images (not just dangling). `docker container prune`: stopped containers. `docker image prune`: dangling images (`-a` for all unused). `docker volume prune`: unused volumes (NOT included in system prune by default — add `--volumes`). `docker network prune`: unused networks. Check disk usage: `docker system df`. Schedule cleanup in cron or CI/CD to prevent disk filling.

Q20. What is a health check in Docker?

■ HEALTHCHECK in Dockerfile defines a command Docker runs periodically to test container health. Example: `HEALTHCHECK --interval=30s --timeout=10s --retries=3 CMD curl -f http://localhost:8080/actuator/health || exit 1`. States: starting (during initial period), healthy (check passes), unhealthy (check fails N times). Docker Swarm restarts unhealthy containers. In Compose: `healthcheck`: with `test`, `interval`, `timeout`, `retries`. `depends_on` with `condition: service_healthy` waits for healthy state before starting dependent services.

Q21. What is Docker in Docker (DinD)?

■ DinD runs a Docker daemon inside a Docker container — enables building Docker images in containerized CI pipelines (Jenkins agent running in Docker). Two approaches: True DinD (privileged container with full Docker daemon — security risk), and Docker socket mounting (mount `/var/run/docker.sock` — gives full host Docker access). Socket mounting is more common but grants significant host privileges. Kaniko and Buildah are rootless, daemon-free alternatives for building images in containers without DinD.

End of Interview Q&A Bank

18 Topics • Full-Stack Development & QA Engineering

Best of luck with your interviews! ■